

复旦大学计算机科学技术学院

《数据结构》期末考试试卷

A 卷 共 7 页

课程代码：INFO130051.02-03      考试形式：□开卷    ☒闭卷      2010 年 1 月  
(本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效)

专业\_\_\_\_\_ 学号\_\_\_\_\_ 姓名\_\_\_\_\_ 成绩\_\_\_\_\_

|    |   |   |   |   |    |
|----|---|---|---|---|----|
| 题号 | 一 | 二 | 三 | 四 | 总分 |
| 得分 |   |   |   |   |    |

一、填空题 (20%)

1、 以下算法的时间复杂度为\_\_\_\_\_。

```
void function(int n)

{

    int i=1; s = 1;

    while(s<n) s += ++i;

    return i;

}
```

2、 对给定的 n 个元素，建立一个有序单链表的时间复杂度是\_\_\_\_\_。

3、 高度为 i (i ≥1) 的完全二叉树最多有\_\_\_\_\_个结点，最少有\_\_\_\_\_个结点。若按从上到下，从左到右顺序从 1 开始给结点编号，则编号最小的叶子结点的编号为\_\_\_\_\_。

4、 有 16 条边的无向图，有 3 个 4 度顶点，有 4 个 3 度顶点，其余顶点的度数都小于 3，则此图至少有\_\_\_\_\_个顶点。

5、 n 个结点和 e 条边构成的无向图的邻接矩阵中，零元素的个数为\_\_\_\_\_。

6、  $n$  个顶点  $e$  条边的图的深度优先遍历的空间复杂度为\_\_\_\_\_。

7、 由 25、10、60、50 构成的二叉排序树有\_\_\_\_\_种；其中 AVL 树有\_\_\_\_\_种。

## 二、选择题（10%）

1、 对于顺序查找，假定查找成功与不成功的概率相同，对每个记录查找的概率也相同，则顺序查找的平均查找长度为\_\_\_\_\_。

- |   |            |   |              |
|---|------------|---|--------------|
| A | $0.5(n+1)$ | B | $0.25(n+1)$  |
| C | $0.5(n-1)$ | D | $0.75n+0.25$ |

2、 下列 算法占用的辅助空间最多。

- A 堆排序                      B 希尔排序                      C 快速排序                      D 归并排序

3、 设有无向图  $G = (V, E)$  和  $G' = (V', E')$ ，如  $G'$  是  $G$  的生成树，则下面错误的说法为\_\_\_\_\_。

- A  $G'$  是  $G$  的连通分量  
B  $G'$  是  $G$  的无环子图  
C  $G'$  是  $G$  的子图  
D  $G'$  是  $G$  的极小连通子图且  $V' = V$

4、 二叉树在线索化后，仍不能有效求解的问题是\_\_\_\_\_。

- A 前序线索化二叉树中求前序后继      B 中序线索化二叉树中求中序后继
- C 中序线索化二叉树中求中序前驱      D 后序线索化二叉树中求后序后继

5、 下列叙述中不正确的是\_\_\_\_\_。

- A 任何一个无向连通图有一棵或者多棵最小生成树。
- B 克鲁斯卡尔算法比较适合边稠密的图，普里姆算法比较适合边稀疏的图。
- C 克鲁斯卡尔算法与普里姆算法均可能生成不同的最小生成树。
- D 有向完全图一定是强连通有向图。

### 三、问答题 (35%)

- 1、已知一个有向网络的邻接矩阵 A 如下，现在要在该网络中的某个结点建立超市，要求该结点距离其它各结点的最长往返路程最短，在最长往返路程相等的情况下，则选择到其它所有结点的往返路程总和最短的结点。问应该选择哪个结点，给出解题过程（结点编号为 1-6）。(10 分)

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | - | 0 | - | 4 | - |
| 1 | - | - | 0 | 1 | - |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

- 2、在 AVL 树中插入一个新结点后，怎么判断需要调整 AVL 树恢复平衡？怎样判断这种调整可以结束，不再需要进一步的调整？(5 分)

3、在  $n$  个不同元素的顺序表中找出最大和最小值需要至少进行多少次比较？算法的思路是怎样的？（6 分）

4、对数据元素 1、13、29、18、75、60、43、54、90、46、21、34，采用散列函数  $\text{hash}(\text{key}) = \text{key} \% 13$  和线性探测再散列方法构造散列表  $\text{HT}[13]$ 。（1）请计算出各元素散列地址。（2）求搜索成功和不成功的平均搜索长度。（6 分）

- 5、若存于顺序存储结构中的  $n$  个元素的序列近似有序，即去掉其中少数  $k$  个元素的序列是有序序列，试对这种数据文件讨论用直接插入排序法、起泡排序法和直接选择排序法的渐进时间复杂度。(8 分)

#### 四、算法题 (35%)

(除写出相应算法之外，还需要说明算法和数据结构的设计思路，算法用 c 或者 C++ 语法描述)

- 1、设  $T$  是一棵满二叉树，写一个递归算法，把  $T$  的前序遍历序列转换成后序遍历序列。(10 分)

- 2、 设  $A$ 、 $B$  是长度为  $n$  的两个非降序数组。如果将这  $2n$  个数全体排序，处于位置  $n$  的数称为中位数。设计一个最坏情况下  $O(\log n)$  的算法，求这个中位数。并且证明你的算法的时间复杂性。  
(15 分)

- 3、设计一个算法，判断一个有向无环图中是否存在这样的顶点，该顶点到其它任意顶点都有一条有向路。(10 分)

# 复旦大学计算机科学技术学院

## 《数据结构》期末考试答案及评分标准

### A 卷 共 7 页

课程代码: INFO130051.02-03 考试形式: ☐ 开卷 ☒ 闭卷

2010 年 1 月

(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

#### 一、填空题 (20%)

- 1、 答:  $O(n^{1/2})$
- 2、 答:  $O(n^2)$
- 3、 答:  $2^{i-1}, 2^{(i-1)}, 2^{(i-2)+1}$
- 4、 答: 11
- 5、 答:  $n^2 - e$
- 6、 答:  $O(n)$
- 7、 答: 14 5 (15? 4?)

#### 二、选择题 (10%)

- 1、 答: d
- 2、 答: d
- 3、 答: a
- 4、 答: d
- 5、 答: b

#### 三、问答题 (35%)

- 1、 已知一个有向网络的邻接矩阵 A 如下, 现在要在该网络中的某个结点建立超市, 要求该结点距离其它各结点的最长往返路程最短, 在最长往返路程相等的情况下, 则选择到其它所有结

点的往返路程总和最短的结点。问应该选择哪个结点，给出解题过程（结点编号为 1-6）。(10 分)

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | - | 0 | - | 4 | - |
| 1 | - | - | 0 | 1 | - |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

答：先用 floyd 算法算出所有结点之间的最短路径，然后对每个结点 i 算出到其它各结点的往返距离总和 sum(i)，以及 i 到其它结点的最长距离 long(i)，最后选择结点 5。

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | 6 | 0 | - | 4 | 7 |
| 1 | 3 | - | 0 | 1 | 4 |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | 6 | 0 | 8 | 4 | 7 |
| 1 | 3 | 6 | 0 | 1 | 4 |
| - | 1 | 4 | 3 | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| 0 | 2 | 5 | 4 | 9 | 3  |
| 7 | 0 | 3 | 2 | 7 | 10 |
| 4 | 6 | 0 | 8 | 4 | 7  |
| 1 | 3 | 6 | 0 | 1 | 4  |
| 8 | 1 | 4 | 3 | 0 | 3  |
| 6 | 8 | 2 | 5 | 6 | 0  |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 6 | 0 | 8 | 4 | 7 |
| 1 | 3 | 6 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 8 | 2 | 5 | 6 | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 5 | 0 | 7 | 4 | 7 |
| 1 | 2 | 5 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 7 | 2 | 5 | 6 | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 5 | 0 | 7 | 4 | 7 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 5 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 7 | 2 | 5 | 6 | 0 |

Sum(1)=2+5+4+5+3+3+4+1+4+6=37

Sum(2)=3+3+2+3+6+2+5+2+1+7=34

Sum(3)=4+5+7+4+7+5+3+5+4+2=46

Sum(4)=4+2+7+3+5+1+2+5+1+4=34

Sum(5)=5+3+4+1+0+6+4+1+4+3+0+3=34

Sum(6)=3+6+7+4+3+0+6+7+2+5+6=49

Long(1)=9; long(2)=13, long(3)=9, long(4)=12, long(5)=9; long(6)=13

评分标准：总分 10 分，思路正确得 5 分，计算正确得 5 分，其它情况酌情扣分。

2、在 AVL 树中插入一个新结点后，怎么判断需要调整 AVL 树恢复平衡？怎样判断这种调整可以结束，不再需要进一步的调整？（5 分）

答：从新结点开始向上追溯，看新结点的双亲结点的平衡因子，如果变为 0，不需继续向上调整，如果变为 1，则需要继续向上追溯，如果变为 2，则需要做调整，因为调整后局部高度恢复到原来的值，所以不需要继续调整。

评分标准：叙述正确得 5 分，其它情况酌情扣分。

3、在 n 个不同元素的顺序表中找出最大和最小值需要至少进行多少次比较？算法的思路是怎样的？（6 分）

答： $3/2 * n - 2$

把表分为两部分，分别找出各部分的最大最小值，递归进行。

也可每次向前循环两个元素，对这两个元素比较一次，得到最大最小值，然后与前面得到的元素比较最大值和最小值。

评分标准：叙述正确得 4 分，计算正确得 2 分，其它情况酌情扣分。

4、对数据元素 1、13、29、18、75、60、43、54、90、46、21、34，采用散列函数  $\text{hash}(\text{key}) = \text{key} \% 13$  和线性探测再散列方法构造散列表 HT[13]。（1）请计算出各元素散列地址。（2）求搜索成功和不成功的平均搜索长度。（6 分）

答：各元素的散列地址为：1（1）、13（0）、29（3）、18（5）、75（10）、60（8）、43（4）、54（2）、90（12）、46（7）、21（9）、34（11）

搜索成功的平均搜索长度为： $(1 * 10 + 2 * 1 + 4 * 1) / 12 = 4/3$

搜索不成功的平均搜索长度为： $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12) / 13 = 6$

评分标准：散列地址计算正确、两种搜索长度计算正确，各得 2 分，其它情况酌情扣分。

- 5、若存于顺序存储结构中的  $n$  个元素的序列近似有序，即去掉其中少数  $k$  个元素的序列是有序序列，试对这种数据文件讨论用直接插入排序法、起泡排序法和直接选择排序法的渐进时间复杂度。（8 分）

答：对于基本有序的  $n$  个元素，用直接插入排序法其渐进时间复杂度为  $O(k*n)$ ，其中  $k$  为排列无序的元素个数， $n$  为排列有序的元素个数。显然，用直接插入法时，对有序元素的插入通常只需要比较 1 次即可确定其位置，而对无序元素的插入至多需要比较  $n$  次；也即，该方法的渐进时间复杂度为  $O(k*n)$ 。在用冒泡排序法、直接选择排序法对基本有序的  $n$  个元素排序时，其渐进时间复杂度都为  $O(n^2)$ 。

评分标准：总分 8 分，各种简单内排序方法分析比较正确或者部分正确为 1-8 分，否则为 0 分。

#### 四、算法题（35%）

（除写出相应算法之外，还需要说明算法和数据结构的设计思路，算法用 c 或者 C++ 语法描述）

- 1、设  $T$  是一棵满二叉树，写一个递归算法，把  $T$  的前序遍历序列转换成后序遍历序列。（10 分）

答：由于是满二叉树，结点总数为  $2n+1$ ，先序的第一个结点为根结点，第 2 到  $n+1$  为左子树的结点，第  $n+2$  到  $2n+1$  为右子树的结点。假设已经有了一个先序数组  $pre[a1,...,b1]$ ，求后序数组  $post[a2,...,b2]$  的算法如下：

```
convert(int[] pre, int[] post, int preStart, int postStart, int k) {
    post[postStart+k] = pre[preStart];
    if(k==0) return;
    convert(pre, post, preStart+1, postStart+k/2+1, k/2);
    convert(pre, post, preStart+k/2+1, postStart, k/2);
    return;
}
```

**评分标准：**总共 10 分。

思路 3 分——思路清晰正确为 3 分，方法不限于参考答案这一种，思路不完全正确最多为 1 分，没写为 0 分；

程序 7 分——程序结构基本正确为 3 分，部分正确为 5 分，全部正确为 7 分。

**注：**由于算法题可能会存在多种求解方法，因此评分过程可依据实际情况进行动态调整。

- 2、设  $A$ 、 $B$  是长度为  $n$  的两个非降序数组。如果将这  $2n$  个数全体排序，处于位置  $n$  的数称为中位数。设计一个最坏情况下  $O(\log n)$  的算法，求这个中位数。并且证明你的算法的时间复杂性。（15 分）

答：可以递归地完成。首先对于两个长度为  $k$  的有序表，可以比较它们各自的中位数  $A[k/2]$  和  $B[k/2]$ ，比较结果分为 3 种情况：

- (1) 如果  $A[k/2] > B[k/2]$ , 则  $A[k/2]..A[k]$  都大于  $B[0]..B[k/2-1]$ , 再加上  $B[k/2]..B[k]$  本来就都大于  $B[0]..B[k/2-1]$ , 多于  $k$  个数比  $B[0]..B[k/2-1]$  大, 所以, 第  $k$  大的数不可能在  $B[0]..B[k/2-1]$  中, 因此可以将  $B[0]..B[k/2-1]$  这部分丢掉; 类似地, 第  $k$  大的数也不可能在  $A[k/2+1]..A[k]$  中, 所以也可以将这部分丢掉, 因此得到了两个新的序列:  $A[0]..A[k/2]$  和  $B[k/2]..B[k]$ , 然后继续递归。
  - (2) 如果  $A[k/2] < B[k/2]$ , 这种情况和第 (1) 种情况类似, 只不过丢掉的是  $A[0]..A[k/2-1]$  和  $B[k/2+1]..B[k]$ , 留下的是  $A[k/2]..A[k]$  和  $B[0]..B[k/2]$ 。
  - (3) 如果  $A[k/2] == B[k/2]$ , 因为  $A[k/2]$  大于  $A[0]..A[k/2-1]$  和  $B[0]..B[k/2-1]$ , 并且  $A[k/2] == B[k/2]$ , 所以第  $k$  大的数就是  $A[k/2]$ 。
- 因为每次比较都能减少一半的元素, 所以最坏情况下递归深度为  $\log n$ , 所以算法的时间复杂度为  $\log n$ 。

算法如下, 假设  $A$  和  $B$  为整型数组:

```
int findMid(int[] a, int[] b, int startA, int startB, int k) {
    if(a[startA+k/2] == b[startB+k/2]) return a[startA+k/2];
    if(a[startA+k/2] > b[startB+k/2]) findMid(a,b,startA+k/2,startB,k/2);
    else (a[startA+k/2] < b[startB+k/2]) findMid(a,b,startA,startB+k/2,k/2);
}
```

**评分标准:** 总共 15 分。

思路 5 分——思路清晰正确为 5 分, 方法不限于参考答案这一种, 思路不完全正确最多为 2 分, 没写为 0 分;

程序 10 分——程序结构基本正确为 5 分, 部分正确为 7 分, 全部正确为 10 分。

**注:** 由于算法题可能会存在多种求解方法, 因此评分过程可依据实际情况进行动态调整。

- 3、设计一个算法, 判断一个有向无环图中是否存在这样的顶点, 该顶点到其它任意顶点都有一条有向路。(10 分)

答: 如果存在这样的顶点, 由于是有向无环图, 则该顶点到其它顶点的路径形成一棵树, 树上的结点数量等于图上的结点数量。因此依次从每个结点开始, 用深度优先或者广度优先, 如果遍历到的结点数量等于图上的结点数量, 则存在, 否则不存在。

如果用广度优先, 图用邻接表表示, 其数据结构描述如下:

```
struct node {
    int adjvex;
    node *next;
}
```

```
struct g {
    vertextype data;
    node *firstarc;
}typedef ADJLIST;
```

```
int rbfs(ADJLIST[] g, int v, int n) {
```

```

int i, count, yes;
yes = 0;
count = 1;
Queue Q;
for(i=0; i<n; i++) visited[i] = 0;
ENQUEUE(v, Q);
visited[v] = 1;

while(!EMPTY(Q)&&!yes) {
    w=Q.front;
    p=g[w].firstarc;
    while((p!=NULL) &&!yes) {
        w=p->adjvex;
        if(visited[w] = 0) {
            visited[w] = 1;
            count++;
            ENQUEUE(w,Q);
        }
        else p=p->next;
    }
}
if(count == n) yes=1;
return yes;
}

Bool  mainLoop(ADJLIST[] g, int n) {
    for(int i=0;i<n;i++)  if( rbfs(g, i, n) == 0) return true;
    return false;
}

```

**评分标准：** 总共 10 分。

思路 2 分——思路清晰正确为 2 分，方法不限于参考答案这一种，思路不完全正确最多为 1 分，没写为 0 分；

关键数据结构 2 分——如邻接表中顶点结点与边结点；

程序 6 分——程序结构基本正确为 2 分，部分正确为 4 分，全部正确为 6 分。

**注：** 由于算法题可能会存在多种求解方法，因此评分过程可依据实际情况进行动态调整。

复旦大学计算机科学技术学院

《数据结构》期中考试试卷

共 7 页

课程代码：COMP130004      考试形式：□开卷    ■闭卷      2011 年 11 月  
(本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效)

专业\_\_\_\_\_学号\_\_\_\_\_班级\_\_\_\_\_姓名\_\_\_\_\_成绩\_\_\_\_\_

|    |   |   |   |    |
|----|---|---|---|----|
| 题号 | 一 | 二 | 三 | 总分 |
| 得分 |   |   |   |    |

一、填空题（25%，第 1~10 题每题 2 分，第 11 题 5 分）

- 1、 一个栈的进栈序列是 A、B、C，写出所有可能的出栈序列\_\_\_\_\_。
- 2、 广义表  $GL = ((a,b,c),(d,e,f))$ ，假设求表头操作为 Head，求表尾操作为 Tail，则  $f =$ \_\_\_\_\_。
- 3、 具有 4 个节点的不同二叉树共有\_\_\_\_\_棵。
- 4、 已知数组  $A[1...10, 1...10]$  为对称矩阵，期中每个元素占 5 个单元。现将其下三角部分按行优先次序存储在起始地址为 1000 的连续内存单元中，则元素  $A[5][6]$  对应的地址为\_\_\_\_\_。
- 5、 算术表达式  $(x+y)/10+a*b/c-(x-y)*(a-b)$  转为后缀表达式后为\_\_\_\_\_。
- 6、 如果表示静态链表的数组  $elem[1].link = 3, elem[1].data = 1, elem[2].link = 3, elem[2].data = 2, elem[3].link = 4, elem[3].data = 5$ ，则在静态链表中  $elem[1]$  指向的下一个元素的值是\_\_\_\_\_。
- 7、 设栈 s 和队列 q 的初始状态为空，元素 a, b, c, d, e 和 f 依次通过栈 s，一个元素出栈后即进入队列 q，若 6 个元素出队的序列是 bdcfea，则栈 s 的容量至少应该存放\_\_\_\_\_个元素。
- 8、 树的后序遍历和它通过左孩子右兄弟方法转换的二叉树的\_\_\_\_\_遍历是等价的。
- 9、 试判断下面的关键码序列中哪一个是堆\_\_\_\_\_。
- ① {94, 31, 53, 23, 16, 72}    ② {16, 31, 23, 94, 53, 72}    ③ {16, 53, 23, 94, 31, 72}
- ④ {94, 53, 31, 72, 16, 23}    ⑤ {16, 72, 31, 23, 94, 53}

10、已知一棵二叉树是以二叉链表的形式存储的，其结点结构说明如下：

```
struct node {  
    int data;  
    struct node *left;  
    struct node *right;  
}
```

请在下面的\_\_\_\_\_处进行填空，空成题目要求的功能。注意：每空只能填一个语句，多填为 0 分。

求出以 T 为根的二叉树或子树的结点个数

```
int size (struct node *T){  
    if (_____  
        return 0;  
    else  
        _____  
}
```

11、高度为 h 的满二叉树(当只有一个根结点时, h 为 0), 其结点总数 n 可以用 h 表示为\_\_\_\_\_, 而 h 可以用 n 表示为\_\_\_\_\_, 若该树的结点从零开始按中序遍历顺序编号, 则树根的编号用 h 表示为\_\_\_\_\_, 树根的左孩子的编号用 h 表示为\_\_\_\_\_, 树根的右孩子的编号用 h 表示为\_\_\_\_\_。

## 二、问答题 (35%, 每题 7 分)

1、一棵二叉树的前序、中序和后序遍历序列分别如下, 其中有一部分未显示出来 (每个空白均只有一个节点), 请补全下列空白并画出该二叉树。

先序序列: \_\_B\_\_F\_\_ICEH\_\_G;

中序序列: D\_\_KFIA\_\_EJC\_\_;

后序序列: \_\_K\_\_FBHJ\_\_G\_\_A;

- 2、已知目标串  $T = \text{"abcabbabcabaaa"}$ ，模式串  $P = \text{"abcabaa"}$ 。
- a) 请按照 KMP 算法的要求计算模式串 P 的  $\text{next}[ ]$  数组(KMP 原型和优化方法二选一即可)。
  - b) 请画出利用 a) 中生成的  $\text{next}$  数组，KMP 算法进行模式匹配时的每一趟匹配过程。

- 3、找出所有满足下列条件的二叉树，假设任意两个节点都不相同：
- (1) 它们的前序遍历序列和中序遍历序列相同。
  - (2) 它们的后序遍历序列和中序遍历序列相同。
  - (3) 它们的前序遍历序列和后序遍历序列相同。

- 4、假设在通信中，只允许出现 8 种字符，分别用 a、b、c、d、e、f、g、h 来代替，假设字符出现的频度依次为 9、17、2、6、22、13、11、20，试建立哈夫曼树。

- 5、有如下递归算法：

```
void print( int w) {  
    int i;  
    if ( w != 0){  
        print(w - 1);  
        for ( i = 1; i <= w; i++)  
            cout<<w;  
        cout<<endl;  
    }  
}
```

调用语句 `print(4)` 的结果是什么？不用栈将该递归算法改为非递归。

### 三、算法设计题 (40%)

(首先用简明的文字写出算法基本思路, 不写思路不评分; 然后给出数据结构和算法的 C 或者 C++ 描述)

- 1、将下面 Hanoi 塔问题的递归算法转化为非递归算法。(10 分)

```
void towers(int n, int A, int B, int C)
{
    if(n == 1) cout<<A<<"→"<<C<<endl;
    else
    {
        towers(n-1, A, C, B);
        cout<<A<<"→"<<C<<endl;
        towers(n-1, B, A, C);
    }
}
```

2、已知二叉树以完全二叉树形式存储在数组当中， $p$ 、 $q$  为二叉树的两个节点，该二叉树的节点个数  $n$ ，设计算法求这两个节点的最近公共祖先节点。(最近公共祖先节点是  $p$  和  $q$  公共祖先当中层数最大的节点) (15 分)

- 3、试编写算法，对二叉树做自底向上，自左向右的层次遍历，并按遍历次序输出各结点的值。  
二叉树采用二叉链表存储。(15 分)

## 《数据结构》期中考

### 一、 填空题 (25%) (第 1~10 题每题 2 分, 第 11 题 5 分)

1. 一个栈的进栈序列是 A、B、C, 写出所有可能的出栈序列  
\_\_\_\_\_。

答案: ABC, ACB, BAC, BCA, CBA

评分标准: 少写或填错不给分, 只有全对才给 2 分。

2. 广义表  $GL = ((a,b,c),(d,e,f))$ , 假设求表头操作为 Head, 求表尾操作为 Tail, 则  
 $f =$ \_\_\_\_\_。

答案: head [ tail [ tail [ head [ tail [ GL ] ] ] ] ]

3. 具有 4 个节点的不同二叉树共有\_\_\_\_\_棵。

答案: 14 或者是  $14 * 4! = 336$

4. 已知数组  $A[1...10, 1...10]$  为对称矩阵, 期中每个元素占 5 个单元。现将其下三角部分按行优先次序存储在起始地址为 1000 的连续内存单元中, 则元素  $A[5][6]$  对应的地址为  
\_\_\_\_\_。

答案: 1095

5. 算术表达式  $(x+y)/10+a*b/c-(x-y)*(a-b)$  转为后缀表达式后为\_\_\_\_\_。

答案:  $xy+10/ab*c/+xy-ab-*-$

6. 如果表示静态链表的数组  $elem[1].link = 3$ ,  $elem[1].data = 1$ ,  $elem[2].link = 3$ ,  
 $elem[2].data = 2$ ,  $elem[3].link = 4$ ,  $elem[3].data = 5$ , 则在静态链表中  $elem[1]$   
指向的下一个元素的值是\_\_\_\_\_。

答案: 5

7. 设栈 s 和队列 q 的初始状态为空, 元素 a, b, c, d, e 和 f 依次通过栈 s, 一个元素出栈后即  
进入队列 q, 若 6 个元素出队的序列是 bdcfea, 则栈 s 的容量至少应该存放\_\_\_\_\_个元素。

答案: 3

8. 树的后序遍历和它通过左孩子右兄弟方法转换的二叉树的\_\_\_\_\_遍历是等价的。

答案: 中序

9. 试判断下面的关键码序列中哪一个是堆\_\_\_\_\_。

- ① {94, 31, 53, 23, 16, 72}    ② {16, 31, 23, 94, 53, 72}    ③ {16, 53, 23, 94, 31, 72}  
④ {94, 53, 31, 72, 16, 23}    ⑤ {16, 72, 31, 23, 94, 53}

答案: 2

10. 已知一棵二叉树是以二叉链表的形式存储的, 其结点结构说明如下:

```
struct node {
```

```

int data;
struct node *left;
struct node *right;
}

```

请在下面的\_\_\_\_\_处进行填空，空成题目要求的功能。注意：每空只能填一个语句，多填为 0 分。

求出以 T 为根的二叉树或子树的结点个数

```

int size (struct node *T){
    if (_____)
        return 0;
    else
        _____
}

```

答案： T == NULL, return(1+size(T->left)+size(T->right));

评分标准：每空 1 分。共 2 分。

11. 高度为 h 的满二叉树(其结点总数 n 可以用 h 表示为\_\_\_\_\_, 而 h 可以用 n 表示为\_\_\_\_\_, 若该树的结点从零开始按中序遍历顺序编号, 则树根的编号用 h 表示为\_\_\_\_\_, 树根的左孩子的编号用 h 表示为\_\_\_\_\_, 树根的右孩子的编号用 h 表示为\_\_\_\_\_。

答案：  $2^{h+1}-1$ ,  $\log_2(n+1)-1$ ,  $2^h-1$ ,  $2^{h-1}-1$ ,  $2^{h+1}-2^{h-1}-1$

评分标准： 每空 1 分，共 5 分。

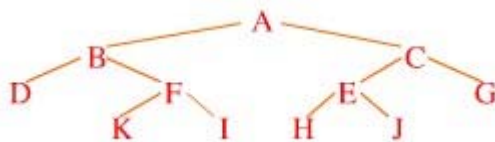
## 二、 问答题（35%，每题 7 分）

1. 一棵二叉树的前序、中序和后序遍历序列分别如下，其中有一部分未显示出来（每个空白均只有一个节点），请补全下列空白并画出该二叉树。（7 分）

先序序列： \_B\_F\_ICEH\_G;

中序序列： D\_KFIA\_EJC\_;

后序序列： \_K\_FBHJ\_G\_A;



答案：

先序序列： ABDFKICEHJG 中序序列：DBKFIAHEJCG 后序序列：DKIFBHJEGCA

评分标准：

**先序** 2 分，每一个空格 0.5 分

**中序** 2 分，只对 1 个给 1 分，对 2 个给 1.5 分，对 3 个给 2 分。

**后序** 2 分，每一个空格 0.5 分

**二叉树** 1 分，画错不给分，全对给 1 分。

2. 已知目标串 T="abcabbabcabaaa", 模式串 P="abcabaa"。(7 分)

a) 请按照 KMP 算法的要求计算模式串 P 的 next[] 数组包括 next 数组定义方法和优化方法。

b) 请画出利用 a) 生成的 next 数组, KMP 算法进行模式匹配时的每一趟匹配过程。

评分标准: a 部分 4 分, 每一空给 0.5 分。全对给 4 分。刘鹏

b 部分 3 分, 对 1 个给 1 分, 对 2 个给 1.5 分, 对 3 个给 2 分, 全对给 3 分。

定义

|    |   |   |   |   |   |   |
|----|---|---|---|---|---|---|
| 0  | 1 | 2 | 3 | 4 | 5 | 6 |
| A  | B | C | A | B | A | A |
| -1 | 0 | 0 | 0 | 1 | 2 | 1 |

优化:

|    |   |   |    |   |   |   |   |
|----|---|---|----|---|---|---|---|
| 0  | 1 | 2 | 3  | 4 | 5 | 6 | 7 |
| A  | B | C | A  | B | A | A |   |
| -1 | 0 | 0 | -1 | 0 | 2 | 1 | 1 |

(1)

定义匹配过程:

abcabbabcabaaa

|||| X

abcabaa

abcabbabcabaaa

|| X

abcabaa

abcabbabcabaaa

X

abcabaa

abcabbabcabaaa

|||||

abcabaa

(2) 优化匹配过程

abcabbabcabaaa

|||| X

abcabaa

abcabbabcabaaa

|| X

abcabaa

abcabbabcabaaa

X

abcabaa

abcabbabcaaaa

|||||

abcabaa

3. 找出所有满足下列条件的二叉树，假设任意两个节点都不相同：

- (1) 它们的前序遍历序列和中序遍历序列相同。
- (2) 它们的后序遍历序列和中序遍历序列相同。
- (3) 它们的前序遍历序列和后序遍历序列相同。

答案 (1) 空二叉树、仅有根节点的二叉树、全部只有右子树的二叉树

(2) 空二叉树、仅有根节点的二叉树、全部只有左子树的二叉树

(3) 空二叉树、仅有根节点的二叉树

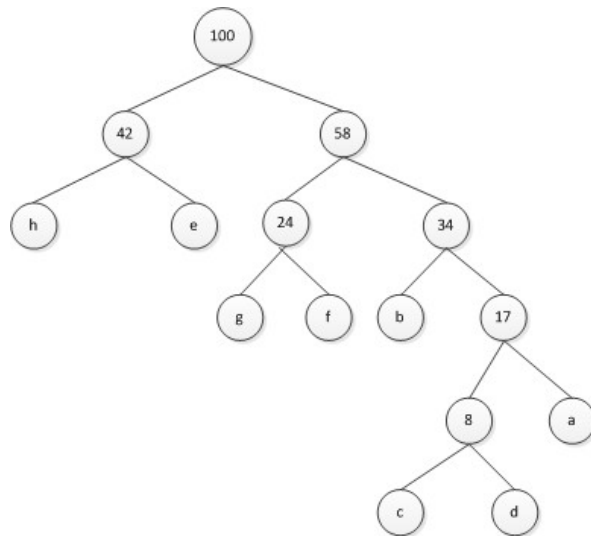
评分标准：刘鹏

(1) 3 分，对一个给 1 分，一共 3 分

(2) 3 分，对一个给 1 分，一共 3 分。

(3) 1 分，对一个给 0.5 分，一共 1 分。

4. 假设在通信中，只允许出现 8 种字符，分别用 a、b、c、d、e、f、g、h、来代替，假设字符出现的频率依次为 9、17、2、6、22、13、11、20，试建立哈夫曼树。



答案：

评分标准：一共有 7 步，每对一步给 1 分。共 7 分。刘鹏

5. 有如下递归算法：

```
void print( int w) {
    int i;
    if ( w != 0){
        print(w - 1);
        for ( i = 1; i <= w; i++)
            cout<<w;
        cout<<endl;
    }
}
```

调用语句 `print(4)` 的结果是什么？不用栈将该递归算法改为非递归。

答案： `print(4)` 的结果是：

```
1
2 2
3 3 3
4 4 4 4
void print( int w) {
    int k = 1;
    while(k!=w)
    {
        For(int i = 1; i <=k; i++)
            cout<<k;
        cout<<endl;
        k++;
    }
}
```

评分标准：1. 输出结果 4 分，每对一行给 1 分。共 4 分。刘鹏

2. 改非递归 3 分。代码基本正确给 3 分，部分有错不影响正常阅读给 2 分，只对部分给 1 分，错误给 0 分）

### 三、 算法设计题（40%）

1、 将下面 Hanoi 塔问题的递归算法转化为非递归算法。（10 分）

```
void towers(int n, int A, int B, int C)
{
    if(n == 1) cout<<A<<"→"<<C<<endl;
    else
    {
        towers(n-1, A, C, B);
        cout<<A<<"→"<<C<<endl;
        towers(n-1, B, A, C);
    }
}
```

参考答案:

评分标准: 思路基本正确(提到用栈来转化成非递归)给 1 分, 正确并详细说明酌情给 2~3 分; 否则本题 0 分。不写思路不评分。

代码基本正确给 7 分, 代码能表达出思路的正确意思但部分书写错误, 每错一处扣一分。代码不知所云给 0 分。 (刘雪君)

```
void towers( int n, int A, int B, int C){
    stack s;
    int done = false;

    while(!done){

        while(n>1){
            s_tack(n, A,B,C,&s);
            setVar1(&n, &A, &B, &C);
        }
        Cout<<A<<"→"<<C<<endl;
        if(!empty(&s))
        {
            Restore(&n, &A, &B, &C, &s);
            Cout<<A<<"→"<<C<<endl;
            setVar2(&n, &A, &B, &C);
        }
        else done = true;
    }
}
```

s\_tack()函数将它的参数放入栈中。setVar1()把 n 设为 n-1,并且交换 C 和 B 的值, setVar2()把 n 设为 n-1,并且交换 A 和 B 的值, restore()负责将局部变量恢复。

- 2、已知二叉树以完全二叉树形式存储在数组当中， $p$ 、 $q$  为二叉树的两个节点及该二叉树的节点个数  $n$ ，设计算法求这两个节点的最近公共祖先节点。(节点各不相同) (最近公共祖先节点是  $p$  和  $q$  公共祖先当中层数最大的节点) (15 分)

思路：以完全二叉树形式保存二叉树，求某结点(数组编号为  $i$ )的祖先，如果此节点的序号  $i$  为奇数，则其父亲结点为  $(i-1)/2$ ，否则为  $(i-2)/2$ 。则祖先节点可以通过此方法求出。求  $p, q$  的公共祖先的第一个交集，即为最近公共祖先。

评分标准：(思路正确清晰 3 分，代码正确完整 12 分)。否则本题 0 分。不写思路不评分。

代码基本正确，缺少对  $p$ 、 $q$  的根节点判断扣一分，求父节点的思路正确计算错误得 4 分，代码能表达出思路的正确意思但部分书写错误，每错一处扣一分。代码不知所云给 0 分。

参考代码：

```
int nearestAncestor( int *a, int p ,int q){
    if(p == 0 || q == 0) cout << "没有最近公共祖先 <<endl;
    int first[100],firstSize = 0;
    int second[100],secondSize = 0;
    // 获得 p 的所有祖先节点//
    int temp = p;
    while(temp!=0){
        if(temp%2 == 0)
            { temp = (temp-2)/2; first[firstSize++] = temp;}
        else
            { temp = (temp-1)/2; first[firstSize++] = temp;}
    }
    temp = q;
    while(temp!=0){
        if(temp%2 == 0)
            { temp = (temp-2)/2;
            int i = 0;
            for(i = 0; i < firstSize; i++) if(first[i] == temp) break;
            if( i != firstSize) break;
            }
    }
    cout<<temp << "是最近公共祖先<< endl;
}
```

3、试编写算法，对二叉树做自底向上，自左向右的层次遍历，并按遍历次序输出各结点的值。(15 分)

思路：要达到自底向上，自左向右的层次遍历，则先自顶向下，自右向左使用队列来对二叉树进行层次遍历，每个出队的节点进入一个栈中，注意在遍历的时候是先遍历右孩子节点再遍历左孩子节点。出栈的时候才能达到自底向上，自左向右的层次遍历结果。

评分标准：思路正确给 5 分。思路中没有提到如何使用栈出队序列达到自左向右的层次遍历的处理，得 3 分。

代码基本正确得 10 分，代码中如果没有实现自左向右（实现了自右向左），得 5 分。

```
void revTraversing (NODE *head){
    queue<NODE *> q;
    stack<Type> data;
    NODE *p = head,*temp;
    q.push(p);
    while( !q.empty()){
        temp = q.pop();
        data.push(temp->data);
        if(temp->rchild!=null) q.push(temp->rchild);
        if(temp->lchild!=null) q.push(temp->lchild);
    }
    While(!data.isEmpty())
    {
        cout<<data.top();
        data.pop();
    }
}
```

#### 详细评分标准（陈楚南）

##### 1. 得分：

- a) 思路正确：3 分
- b) 实现层次遍历：4 分
- c) 自左向右输出：8 分

##### 2. 扣分：

- a) Root 没有判断是否为 null：扣 1 分
- b) 输出顺序是自右向左：扣 5 分
- c) 输出树节点而不是 data：扣 1 分

复旦大学计算机科学技术学院

《数据结构》期中考试试卷

共? 页

课程代码：COMP130004.01-03    考试形式：□开卷    □闭卷    2012 年 11 月  
(本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效)

专业\_\_\_\_\_ 学号\_\_\_\_\_ 姓名\_\_\_\_\_ 成绩\_\_\_\_\_

|    |   |   |   |    |
|----|---|---|---|----|
| 题号 | 一 | 二 | 三 | 总分 |
| 得分 |   |   |   |    |

一、填空题（20%）

- 1、 假设字符串下标从 1 开始，模式串 P="abaabcac"的 next 函数值（未优化）序列为\_\_\_\_\_。[答：-1、0、0、1、1、2、0、1]。
- 2、 使用一个 100 个元素的数组存储循环队列，如果采取少用一个元素空间的方法来区别循环队列的队空和队满，约定队头指针 front 等于队尾指针 rear 时表示队空。若为 front=8，rear=7，则队列中的元素个数为\_\_\_\_\_。[答：99]。
- 3、 高已知广义表 A=((a,b),(c,d))，假设求表头操作为 Head，求表尾操作为 Tail，则 Tail(Head(Tail(A)))=\_\_\_\_\_。[答：(d)]。
- 4、 深度为 5 的二叉树至多有\_\_\_\_\_个结点。
- 5、 线索化二叉树的结点 t，如果 t->ltag==0，则前驱或者为\_\_\_\_\_，或者为当前结点左子树中序下的最后一个结点。[答：NULL]。
- 6、 设森林中有 4 棵树，树中结点的个数依次为 n1、n2、n3、n4，则把森林转换成二叉树后，其根结点的右子树上有\_\_\_\_\_个结点。[答：n2+n3+n4]
- 7、 具有 10 个叶子结点的二叉树中，度为 2 的结点有 \_\_\_\_\_个。[答：9]
- 8、 已知序列 25, 13, 10, 12, 9 是最大堆，在序列尾部插入新元素 18，将其再调整为最大堆，调整过程中元素之间进行的比较次数是。[答： 2 ]
- 9、 程序段{i=1;        while (i<=n)    i=i\*3;}的时间复杂度是：\_\_\_\_\_。[答： O(log3n)]
- 10、 设串的长度为 n，则它的子串个数为\_\_\_\_\_。[答： n(n+1)/2 ]

二、问答题（30%）

1、在表达式中，有的运算符要求从右到左计算，如  $A^B^C$  的计算次序应为  $(A^B)^C$ ，这在由中缀生成后缀的算法中是怎样实现的？以表达式  $A^B^C$  为例说明转换的过程。（6分）

答：采用常规的中缀表达式转为后缀表达式的规则，不同之处是对运算符<sup>^</sup>优先级的规定。在算术运算中，先乘除后加减，先括号内后括号外，相同级别的运算符按从左到右的规则运算。而对<sup>^</sup>运算符，其优先级同常规理解，即高于加减乘除而小于左括号。为了适应本题中“从右到左计算”的要求，规定栈顶运算符<sup>^</sup>的级别小于正从表达式中读出的运算符<sup>^</sup>，即刚读出的运算符<sup>^</sup>级别高于栈顶运算符<sup>^</sup>，因此也入栈。

下面以  $A**B**C$  为例说明实现过程：读入 A，不是操作符，直接写入结果表达式。再读入<sup>^</sup>，与运算符栈顶比较（运算符栈顶初始化后，首先压入‘#’作为开始标志），其级别高于‘#’，入栈。再读入 B，直接进入结果表达式。接着读入<sup>^</sup>，与栈顶比较，均为<sup>^</sup>，由于后读入的<sup>^</sup>级别高于栈顶的<sup>^</sup>，因此<sup>^</sup>入栈。接着读入 C，直接到结果表达式。现在的结果（后缀）表达式是 ABC。最后读入‘#’，表示输入表达式结束，这时运算符栈中从栈顶到栈底有两个<sup>^</sup>和一个‘#’。两个运算符<sup>^</sup>退栈至结果表达式，结果表达式变为  $ABC****$ 。运算符栈中只剩‘#’，退栈，运算结束。

2、算法填空：下面给出了建立二叉树的算法，请阅读此算法并把缺失的语句补上（10分）

```
typedef struct BinTreeNode {
    DataType data;
    struct BinTreeNode * leftChild, * rightChild;
}

template <class Type> class Stack {
    ...
public:
    Stack() { ... }           //构造函数
    ~Stack() { ... }
    int push(Type x) { ... }
    Type* pop() { ... }
    Type* getTop() { ... }
    int makeEmpty() { ... }
}

void CreatBinTree(BinTreeNode *&BT,char ls){
    Stack<BinTreeNode*> s;
    s.makeEmpty();
    BT=NULL; //置空二叉树
    BinTreeNode *p;
    int k; istream ins(ls); //把串 ls 定义为输入字符串流对象 ins;
```

```
char ch; ins>>ch; //从 ins 顺序读入一个字符
while (ch != '#') { //逐个字符处理, 直到遇到 '#' 为止, 假设输入的字符串流格式正确
    switch(ch){
        case '(': __ (1) __; k=1; break;
        case ')': s.pop(); break;
        case ',': __ (2) __; break;
        default :
            p=new BinTreeNode;
            __ (3) __;
            p->leftChild=NULL;
            p->rightChild=NULL;
            if(BT==NULL) __ (4) __;
            else if (k==1) (s.getTop())->leftChild=p;
                else (s.getTop())->rightChild=p;
        }
        __ (5) __;
    }
}
```

答: (1)s.push(p) (2)k=2 (3)p->data=ch (4)BT=p (5) ins>>ch

3、试找出分别满足下列条件的所有二叉树:

- (1) 前序序列和中序序列相同。
- (2) 中序序列和后序序列相同。
- (3) 前序序列和后序序列相同。

(6 分)

答:

- (1) 空二叉树、只有一个根结点的二叉树和右斜树。
- (2) 空二叉树、只有一个根结点的二叉树和左斜树。
- (3) 空二叉树、只有一个根结点的二叉树

- 4、已知某完全  $k$  叉树只有度为  $k$  的结点及叶结点，设叶结点数为  $n_0$ ，求树的高度  $h$ 。(8 分)

答：显然可以算出树的结点数为  $n = (k * n_0 - 1) / (k - 1)$

由于完全  $k$  叉树， $(k^{(h-1)} - 1) / (k - 1) < n \leq (k^h - 1) / (k - 1)$

最后算出来  $h$  是以  $k$  为底对  $(k * n_0)$  取对数的上界，或者是以  $k$  为底对  $(k * n_0)$  取对数的下界加 1。

### 三、算法设计题 (50%)

(以下题目，首先用简明的文字写出算法基本思路，然后给出数据结构和算法的 C 或者 C++ 描述，假设已经有现成的栈 Stack 和队列 Queue 的数据结构可以直接使用)

- 1、编写算法计算给定二叉树中叶结点的个数。其中树结点定义如下

```
typedef struct BiTNode{
    DataType data;
    Struct BiTNode *LChild, *RChild;
}BiTNode, *BiTree;。
```

要求：(1) 描述该算法的基本设计思想；(2) 给出程序代码 (10 分)

答：

(1) 算法的基本思想：先序(或中序或后序)遍历二叉树，在遍历过程中查找叶子结点，并计数。由此，需在遍历算法中增添一个“计数”的参数，并将算法中“访问结点”的操作改为：若是叶子，则计数器增 1。

```
(2) void CountLeaf (BiTree T, int& LeafNum) {
    if (T) {
        if ((!T->lchild) && (!T->rchild)) LeafNum++; // 对叶子结点计数
        else {
            CountLeaf(T->lchild, LeafNum); // 求左子树叶子数
            CountLeaf(T->rchild, LeafNum); // 求右子树叶子数
        }
    }
} // CountLeaf
```

2、求递归函数 F(n)的非递归算法：（15 分）

$$F(n)=\begin{cases} n+1 & n=0 \\ n \cdot F\left(\frac{n}{2}\right) & n>0 \end{cases}$$

答：#include <iostream.h>

#define N 20

int main(){

int I; int a[N]; int n;

cout<<"请输入 n:";<<cin>>n;

for(i=0;i<n+1;i++){ if(i<1) a[i]=1; else a[i]=i\*a[i/2]; }

cout<<a[n]<<endl;

return 0;

}

3、求子数组的最大和：输入一个整形数组，数组里有正数也有负数。数组中连续的一个或多个整数组成一个子数组，每个子数组都有一个和。求所有子数组的和的最大值。要求时间复杂度为  $O(n)$ 。例如输入的数组为 1, -2, 3, 10, -4, 7, 2, -5，和最大的子数组为 3, 10, -4, 7, 2，因此输出为该子数组的和 18。要求：（1）描述该算法的基本设计思想；（2）给出程序代码（15 分）

答：(1)采用贪心策略，从左向右扫描，记录当前的和 sum，当  $sum < 0$  时，把 sum 重置为 0。

```
(2)int maxSubarray(int a[], int size) {  
    if (size <= 0) error("error array size");  
    int sum = 0;  
    int max = - (1 << 31);  
    int cur = 0;  
    while (cur < size) {  
        sum += a[cur++];  
        if (sum > max) {  
            max = sum;  
        } else if (sum < 0) {  
            sum = 0;  
        }  
    } return max;  
}
```

4、从网络上接收类型为 `DataType` 的数据构成的串，每个串的长度事先不确定，但每个串最长不超过 `max` 个数据，接收到的数据放在一个单链表中，由于系统设计的限制，内存中的链表长度不能超过 $\lceil \text{max}/2 \rceil$ ，超过这个长度，则链表第一个结点的数据被丢弃，同时把最新接收的数据作为最后一个结点加入链表中，这些工作都由一个已经开发好的程序 A 完成。单链表结点结构为：

```
typedef struct LNode{
    DataType data;
    Struct LNode *link;
}。
```

请写出算法 `balance(LNode *list)` 判断每个串是否中心对称，例如 `xyx`, `xyyx` 都是中心对称，`list` 是由程序 A 提供的每次开始接收一个串时指向链表第一个结点的指针，限制 `balance` 的空间复杂度不超过  $\text{sizeof(DataType)} * \lceil \text{max}/2 \rceil$ ，可以用 `a1==a2` 来判断 `DataType` 类型的数据 `a1` 和 `a2` 是否相等。要求：（1）描述 `balance` 算法的基本设计思想；（2）写出 `balance` 的程序代码。（10 分）

答：（1）首先要判断链表的中点，用 2 个指针 `p` 和 `q`，开始时都指向第一个结点，以后每次 `p` 前进 2 步，`q` 前进 1 步，`p` 到达链表末尾时，`q` 恰好在中点。其次是判断对称，可以用一个栈，`q` 每次经过的节点内容就入栈，`p` 到达链表末尾时，根据链表是偶数还是奇数长度调整 `q` 后，`q` 继续逐个结点向前，同时开始弹栈，每个弹出的结点与 `q` 经过的结点进行比较。

```
(2) bool balance(LNode *list) {
    DataType stack[max];
    int top=0;
    LNode *p, *q;
    p = q = list;
    while(p) {
        p=p->link;
        if(p) {
            p=p->link;
            stack[top++] = q->data;
            q=q->link;
        }
        else q=q->link; //跳过中间点
    }
    if(p==q) return true; //空表
    if(top==0) return true; //只有一个结点
    while(top>0) {
        if(stack[--top] == q->data) q=q->link;
        else return false;
    }
    return true;
}
```

(装订线内不要答题)

# 复旦大学计算机科学技术学院

## 《数据结构》期中考试试卷

共? 页

课程代码: COMP130004.01-03    考试形式: ☐开卷    ☐闭卷    2012 年 11 月  
(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

专业\_\_\_\_\_ 学号\_\_\_\_\_ 姓名\_\_\_\_\_ 成绩\_\_\_\_\_

|    |   |   |   |    |
|----|---|---|---|----|
| 题号 | 一 | 二 | 三 | 总分 |
| 得分 |   |   |   |    |

### 一、填空题 (20%)

- 1、假设字符串下标从 1 开始, 模式串  $P = \text{"abaabcac"}$  的 next 函数值 (未优化) 序列为\_\_\_\_\_。
- 2、使用一个 100 个元素的数组存储循环队列, 如果采取少用一个元素空间的方法来区别循环队列的队空和队满, 约定队头指针 front 等于队尾指针 rear 时表示队空。若为 front=8, rear=7, 则队列中的元素个数为\_\_\_\_\_。
- 3、高已知广义表  $A = ((a,b),(c,d))$ , 假设求表头操作为 Head, 求表尾操作为 Tail, 则  $\text{Tail}(\text{Head}(\text{Tail}(A))) = \text{\_\_\_\_\_\_}$ 。
- 4、深度为 5 的二叉树至多有\_\_\_\_\_个结点。
- 5、线索化二叉树的结点 t, 如果  $t \rightarrow \text{ltag} = 0$ , 则前驱或者为\_\_\_\_\_, 或者为当前结点左子树中序下的最后一个结点。
- 6、设森林中有 4 棵树, 树中结点的个数依次为  $n_1, n_2, n_3, n_4$ , 则把森林转换成二叉树后, 其根结点的右子树上有\_\_\_\_\_个结点。
- 7、具有 10 个叶子结点的二叉树中, 度为 2 的结点有 \_\_\_\_\_ 个。
- 8、已知序列 25, 13, 10, 12, 9 是最大堆, 在序列尾部插入新元素 18, 将其再调整为最大堆, 调整过程中元素之间进行的比较次数是\_\_\_\_\_。
- 9、程序段 {i=1;        while (i<=n)    i=i\*3;} 的时间复杂度是: \_\_\_\_\_。
- 10、设串的长度为 n, 则它的子串个数为\_\_\_\_\_。

## 二、问答题 (30%)

- 1、在表达式中，有的运算符要求从右到左计算，如  $A^B^C$  的计算次序应为  $(A^{(B^C)})$ ，这在由中缀生成后缀的算法中是怎样实现的？以表达式  $A^B^C$  为例说明转换的过程。(6 分)
- 2、算法填空：下面给出了建立二叉树的算法，请阅读此算法并把缺失的语句补上 (10 分)

```
typedef struct BinTreeNode {
    DataType data;
    struct BinTreeNode * leftChild, * rightChild;
}

template <class Type> class Stack {
    ...
public:
    Stack() { ... }           //构造函数
    ~Stack() { ... }
    int push(Type x) { ... }
    Type* pop() { ... }
    Type* getTop() { ... }
    int makeEmpty() { ... }
}

void CreatBinTree(BinTreeNode *&BT, char ls) {
    Stack<BinTreeNode*> s;
    s.makeEmpty();
    BT=NULL; //置空二叉树
    BinTreeNode *p;
    int k; istream ins(ls); //把串 ls 定义为输入字符串流对象 ins;
    char ch; ins>>ch; //从 ins 顺序读入一个字符
    while (ch != '#') { //逐个字符处理，直到遇到 '#' 为止，假设输入的字符串流格式正确
        switch(ch) {
            case '(': __ (1) __; k=1; break;
            case ')': s.pop(); break;
            case ',': __ (2) __; break;
            default :
                p=new BinTreeNode;
                __ (3) __;
                p->leftChild=NULL;
                p->rightChild=NULL;
                if(BT==NULL) __ (4) __;
                else if (k==1) (s.getTop())->leftChild=p;
                else (s.getTop())->rightChild=p;
        }
        __ (5) __;
    }
}
```

```
    }
}
```

3、试找出分别满足下列条件的所有二叉树：

- (1) 前序序列和中序序列相同。
- (2) 中序序列和后序序列相同。
- (3) 前序序列和后序序列相同。

(6 分)

4、已知某完全 k 叉树只有度为 k 的结点及叶结点，设叶结点数为  $n_0$ ，求树的高度 h。(8 分)

### 三、算法设计题 (50%)

(以下题目，首先用简明的文字写出算法基本思路，然后给出数据结构和算法的 C 或者 C++描述，假设已经有现成的栈 Stack 和队列 Queue 的数据结构可以直接使用)

1、编写算法计算给定二叉树中叶结点的个数。其中树结点定义如下

```
typedef struct BiTNode{
    DataType data;
    Struct BiTNode *LChild, * RChild;
}BiTNode, *BiTree;。
```

要求：(1) 描述该算法的基本设计思想；(2) 给出程序代码 (10 分)

2、求递归函数  $F(n)$  的非递归算法：(15 分)

$$F(n) = \begin{cases} n+1 & n=0 \\ n \cdot F\left(\frac{n}{2}\right) & n>0 \end{cases}$$

3、求子数组的最大和：输入一个整形数组，数组里有正数也有负数。数组中连续的一个或多个整数组成一个子数组，每个子数组都有一个和。求所有子数组的和的最大值。要求时间复杂度为  $O(n)$ 。例如输入的数组为 1, -2, 3, 10, -4, 7, 2, -5，和最大的子数组为 3, 10, -4, 7, 2，因此输出为该子数组的和 18。要求：(1) 描述该算法的基本设计思想；(2) 给出程序代码 (15 分)

4、从网络上接收类型为 DataType 的数据构成的串，每个串的长度事先不确定，但每个串最长不超过 max 个数据，接收到的数据放在一个单链表中，由于系统设计的限制，内存中的链表长度不能超过  $\lceil \max/2 \rceil$ ，超过这个长度，则链表第一个结点的数据被丢弃，同时把最新接收的数据作为最后一个结点加入链表中，这些工作都由一个已经开发好的程序 A 完成。单链表结点结构为：

```
typedef struct  LNode{
    DataType data;
```

Struct LNode \*link;

}。

请写出算法 `balance(LNode *list)` 判断每个串是否中心对称，例如 `xyx`, `xyyx` 都是中心对称，`list` 是由程序 A 提供的每次开始接收一个串时指向链表第一个结点的指针，限制 `balance` 的空间复杂度不超过 `sizeof(DataType)*⌈max/2⌉`，可以用 `a1==a2` 来判断 `DataType` 类型的数据 `a1` 和 `a2` 是否相等。要求：（1）描述 `balance` 算法的基本设计思想；（2）写出 `balance` 的程序代码。（10 分）

# 复旦大学计算机科学技术学院

## 《数据结构》期末考试试卷（参考答案与评分标准）

### A 卷 共 8 页

课程代码: COMP130004.01-03 考试形式: ☐开卷 ☒闭卷 2012 年 1 月  
(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

专业\_\_\_\_\_学号\_\_\_\_\_姓名\_\_\_\_\_成绩\_\_\_\_\_

| 题号 | 一 | 二 | 三 | 四 | 总分 |
|----|---|---|---|---|----|
| 得分 |   |   |   |   |    |

#### 一、填空题 (25%,1~8 题每题 2 分)

- 1、 设  $W$  为一个二维数组, 其每个数据元素占用 4 个字节, 行下标  $i$  从 0 到 7, 列下标  $j$  从 0 到 3, 则二维数组  $W$  的数据元素共占用\_\_\_\_\_个字节。若按行顺序存放二维数组  $W$ , 其起始地址为 100 (10 进制), 则二维数组元素  $W[6, 3]$  的起始地址为\_\_\_\_\_ (10 进制表示)。

答: 128, 208。

- 2、 后缀算式  $9\ 2\ 3\ +\ -\ 10\ 2\ /\ -$  的值为\_\_\_\_\_。中缀算式  $(3+4X) - 2Y/3$  对应的后缀算式为\_\_\_\_\_。

答: -1,  $3\ 4\ X\ * +\ 2\ Y\ * 3\ /\ -$ 。

- 3、 由权值分别为 11, 8, 6, 2, 5 的叶子结点生成一棵哈夫曼树, 它的带权路径长度为\_\_\_\_\_。

答: 71

- 4、 在有序表 (12, 24, 36, 48, 60, 72, 84) 中二分查找关键字 72 时所需进行的关键字比较次数为\_\_\_\_\_。

答: 2

- 5、 在含  $n$  个顶点和  $e$  条边的无向图 (无自环、重边) 的邻接矩阵中, 零元素的个数为\_\_\_\_\_。

答:  $n^2 - 2e$

- 6、 已知一棵完全二叉树中共有 768 结点, 则该树中共有\_\_\_\_\_个叶子结点。

答: 384

- 7、 有向图的边集为  $\{(a, c), (a, e), (e, b), (e, d), (b, d), (d, c), (c, f)\}$ , 该图的一个拓扑排序为:\_\_\_\_\_

答: a e b d c f

- 8、 当输入序列局部有序或元素个数较小时, 在快速排序、选择排序、插入排序、归并排序、堆

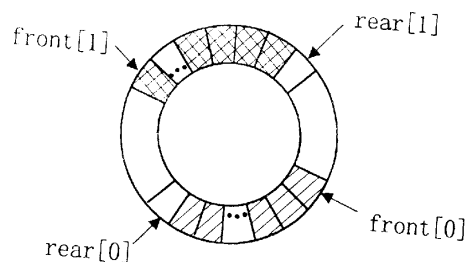
排序中，最佳的排序方法是\_\_\_\_\_。

答：插入排序

9、假设两个队列共享一个循环向量空间（参见右图），

其类型 Queue2 定义如下：

```
typedef struct{
    DataType data[MaxSize];
    int front[2],rear[2];
}Queue2;
```



对于  $i=0$  或  $1$ ， $front[i]$ 和  $rear[i]$ 分别为第  $i$  个队列的头指针和尾指针。请对以下算法填空，实现第  $i$  个队列的入队操作。

```
int EnQueue (Queue2 *Q, int i, DataType x)
{//若第 i 个队列不满，则元素 x 入队列，并返回 1；否则返回 0
    if(i<0||i>1)return 0;
    if(Q->rear[i] == Q->front[_____]) return 0;
    Q->data[_____] = x;
    Q->rear[i] = [_____];
    return 1;
}
```

答：  $(i+1)\%2$  (或  $1-i$ )；  $Q->rear[i]$ ；  $(Q->rear[i]+1)\%Maxsize$ ；（每空 1 分）

10、以下是一个判断二叉树是否平衡的程序，对于平衡的二叉树， $depth$  将会返回其高度（对于非平衡二叉树不要求返回）。请在空白处填上代码完成程序。（6 分）

```
bool isBalanced( BiTreeNode * pRoot, int& depth)
{
    if(pRoot == NULL) {_____; return _____;}
    int leftDepth, rightDepth;
    if(_____)
    {
        int dif = leftDepth - rightDepth;
        if(_____)
        {
            depth = (1+leftDepth) > (1+ rightDepth) ? 1+leftDepth : 1+ rightDepth;
            return _____;
        }
    }
    return false;
}
```

}

答: depth = 0 (1分) true(1分)

isBalanced(pRoot->left, leftDepth) && isBalanced(pRoot->right, rightDepth) (2分)

dif >= -1 && dif <= 1 (1分)

true (1分)

## 二、选择题 (10%)

1、对于双向循环链表,每个结点有两个指针域 next 和 prior,分别指向前驱和后继。在 p 指针所指向的结点之后插入 s 指针所指结点的操作应为 ( )

A. p->next = s; s->prior = p; p->next->prior = s; s->next = p->next;

B. p->next = s; p->next->prior = s; s->prior = p; s->next = p->next;

C. s->prior = p; s->next = p->next; p->next = s; p->next->prior = s;

D. s->prior = p; s->next = p->next; p->next->prior = s; p->next = s;

答: D。

2、一个栈的输入序列为 1 2 3,则下列序列中不可能是栈的输出序列的是( )

A. 2 3 1

B. 3 2 1

C. 3 1 2

D. 1 2 3

答: C。

3、采用开放定址法处理散列表的冲突时,其平均查找长度 ( )。

A. 低于链接法处理冲突

B. 高于链接法处理冲突

C. 与链接法处理冲突相同

D. 高于二分查找

答: B。

4、假设一个有 n 个顶点和 e 条弧的有向图用邻接表表示,则删除与某个顶点  $v_i$  相关的所有弧的时间复杂度是( )

A.  $O(n)$

B.  $O(e)$

C.  $O(n+e)$

D.  $O(n*e)$

答: C。

5、设有 6 个结点的无向图,该图至少应有( )条边才能确保是一个连通图。

A.5

B.6

C.7

D.8

答: A

## 三、问答题 (40%)

1、设一棵 m 叉树中有  $N_1$  个度数为 1 的结点,  $N_2$  个度数为 2 的结点, ...,  $N_m$  个度数为 m 的结点,则该树中共有多少个叶子结点? (请写出求解过程) (5 分)

**答：**度数为  $i$  的结点的孩子个数为  $i \times N_i$ 。除了根结点，其他结点都是某结点的孩子，且父亲结

点唯一。因此该树共有  $\text{total} = 1 + \sum_{i=1}^m i \times N_i$  个结点。

度数大于 0 的结点都是非叶子结点，因此内部结点个数  $\text{inner} = \sum_{i=1}^m N_i$ 。

叶子结点个数  $\text{leaf} = \text{total} - \text{inner} = 1 + \sum_{i=1}^m (i-1) \times N_i = 1 + \sum_{i=2}^m (i-1) \times N_i$

**评分标准：**总分 5 分，分析过程正确或者部分正确为 1-5 分，否则为 0 分。

2、一个线性表为  $B = (12, 23, 45, 57, 20, 03, 78, 31, 15, 36)$ ，设散列表为  $\text{HT}[0..12]$ ，散列函数为  $H(\text{key}) = \text{key} \% 13$  并用线性探查法解决冲突，请画出散列表，并计算等概率情况下查找成功的平均查找长度。（5 分）

答：

| 0  | 1 | 2  | 3  | 4 | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 |
|----|---|----|----|---|----|----|----|----|---|----|----|----|
| 78 |   | 15 | 03 |   | 57 | 45 | 20 | 31 |   | 23 | 36 | 12 |

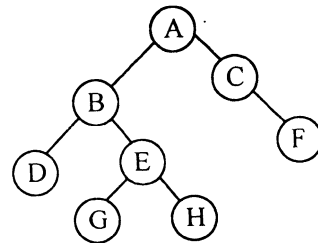
查找成功的平均查找长度： $\text{ASL}_{\text{succ}} = 14/10 = 1.4$

（散列表 4 分，查找成功的平均查找长度 1 分）

3、已知二叉树的存储结构为二叉链表，阅读下面算法。

```
typedef struct node {
    DataType data;
    Struct node * next;
}ListNode;
typedef ListNode * LinkList;
LinkList Leafhead = NULL;
Void Inorder (BinTree T){
    LinkList s;
    If(T){
        Inorder(T->lchild);
        If ((!T->lchild)&&(!T->rchild)){
```

```
s=(ListNode*)malloc(sizeof(ListNode));
s->data=T->data;
s->next=Leafhead;
Leafhead=s;
}
Inorder(T->rchild);
}
```

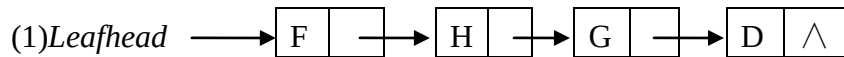


对于如下所示的二叉树

(1) 画出执行上述算法后所建立的结构；(4 分)

(2) 说明该算法的功能。(2 分)

答:

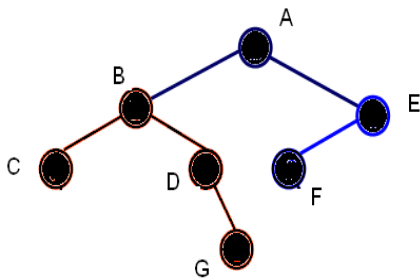


(2) 中序遍历二叉树，按遍历序列中叶子结点数据域的值构建一个以 Leafhead 为头指针的逆序单链表（或按二叉树中叶子结点数据自右至左链接成一个链表）。

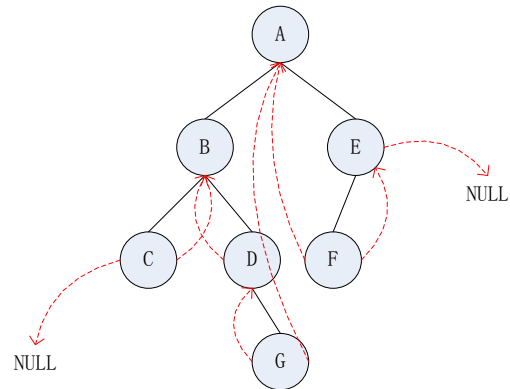
4、一棵二叉树的先序序列为 ABCDGEF，中序序列为 CBDGAFE。请画出该二叉树并将其中序线索化，后将二叉树转换为相应的森林。(5 分)

答:

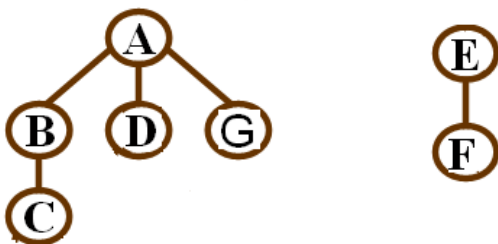
(二叉树: 1 分):



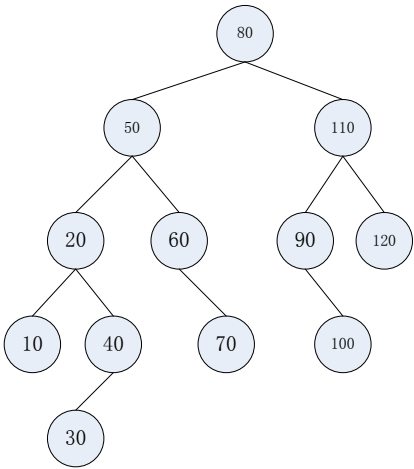
(中序线索: 2 分):



(森林: 2 分):

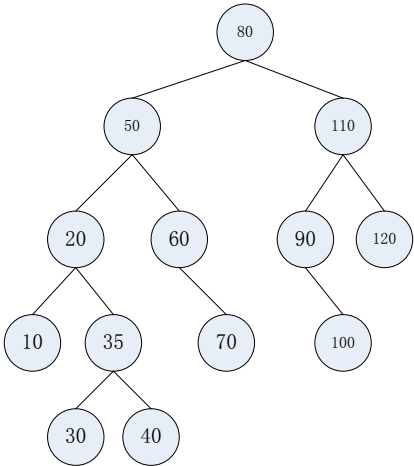


5、对于以下 AVL 树，依次插入关键码 35、28、5。请画出每插入一个关键码之后 AVL 树的形状。  
(6 分)

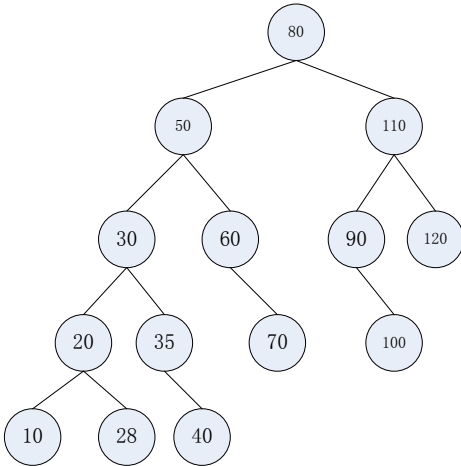


答:

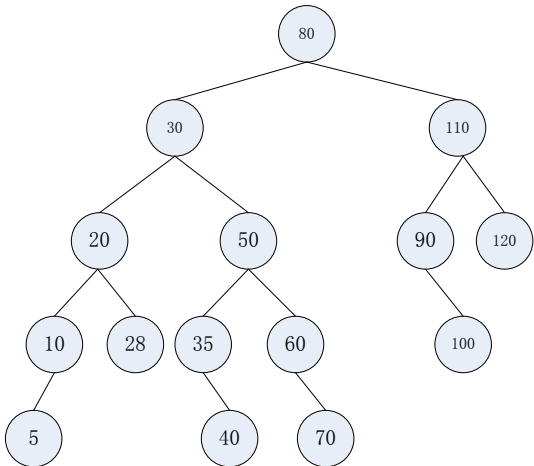
插入 35 (2 分):



插入 28 (2 分):



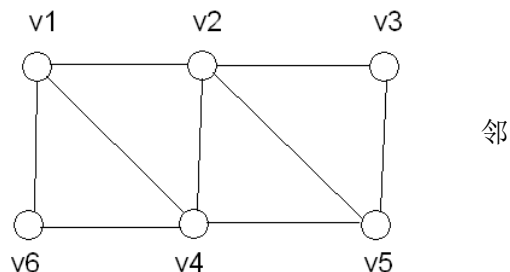
插入 5 (2 分):



**评分标准：**总分 6 分，每次插入后的形状 2 分。

6、已知一个连通图如下图所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点 v1 出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。(8 分)

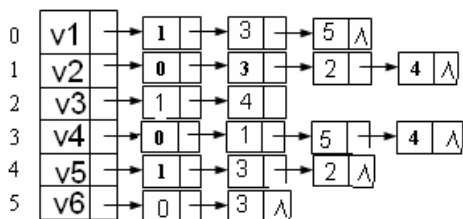
- (1) 图的邻接矩阵 (1 分)
- (2) 邻接表存储示意图 (1 分)
- (3) 从 v1 开始的深度优先遍历的顶点序列 (2 分)
- (4) 分析在深度遍历过程中，分别使用邻接矩阵和接表存储的算法复杂度 (2 分)
- (6) 讨论在图遍历问题中，这两种存储方式的优劣。(2 分)



**答：**(1)图的邻接矩阵 (1 分)

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

(2) 邻接表存储示意图 (1 分)



(3) 深度优先遍历的顶点序列: v1,v2,v3, v5,v4, v6 (2 分) (有多个答案)

(4) 深度优先邻接矩阵复杂度:  $O(n^2)$ , n 为顶点数; 邻接表复杂度:  $O(n+e)$ , n 为顶点数, e 为边的条数。(2 分)

(5)在图的遍历算法中，遍历一个顶点的邻居，对于邻接矩阵，时间复杂度为  $O(n)$ ，而邻接表只需要该顶点的邻居数量复杂度；因此对于整个图的遍历，如深度优先、广度优先等，邻接矩阵需要  $O(n^2)$ ，而邻接表时间复杂度为  $O(n+e)$ ，当图为稀疏图时，邻接表的时间复杂度优于邻接矩阵，而且存储开销也优于邻接矩阵；当图为密集图时，两者性能差不多；总的来说，邻接表的时间复杂度和空间存储开销都优于邻接矩阵。(2 分)

7、分析对比 AVL 树和 Hash 的时空特性，并讨论它们的适合场合。(6 分)

**答：**

时空特性:

AVL 树是高度平衡的二叉查找树, 查找时间复杂度为  $O(\log n)$ , Hash 的查找时间复杂度为  $O(1)$ 。  
存储开销 Hash 往往比 AVL 树小。

适合场合:

Hash 必须处理冲突, 而 AVL 树不存在这种问题。对于删除操作, AVL 树的时间开销很稳定, 为  $O(\log n)$ , 而 Hash, 如果采用拉链法处理冲突, 则删除操作易于实现, 如果采用开放定址法, 则删除操作很难实现, 因此开放定址法的 Hash 不适合更新频繁的数据存储。另外, AVL 树对数据的存储是有序的, 而 Hash 对数据的存储并不能反映数据之间的大小关系。因此, AVL 树适用于有序的数据存储, 而 Hash 适用于数据量比较大且分布比较均匀, 对数据排序无要求的数据存储。

#### 四、算法题 (25%, 第一题要求写代码; 后两题要求写伪代码或步骤清晰的解题思路)

1、堆是一种很常用的数据结构, 其中的一个重要用途是优先队列。以整数最小堆为例, 写出优先队列 (最小优先) 的两个基本操作的代码: (10 分)

(1) 入队, 即插入一个元素到存放堆的数组的末尾, 然后调整堆。

(2) 出队, 即删除堆顶节点, 然后调整堆。

void insert( int k, int a[], int n ) { // 假设新的元素 k 先被放在堆末尾位置 a[n] 单元中

```
    答: n++;
        int j = n-1;
        int i = (j-1)/2;
        while(j > 0)
        {
            if(a[i] <= k)
                break;
            a[j] = a[i];
            j = i;
            i = (i-1)/2;
        }
        a[j] = k;
```

}

int deleteMin( int a[], int n ) { // 假设在堆顶元素被删除之前, 堆中共有 n 个元素

```
    答: a[0] = a[n-1];
        n--;
        int i = 0;
        int temp = a[i];
        while((j = 2*i + 1) < n)
        {
            if( j < n-1 && a[j+1] < a[j])
                j++;
            if(a[j] >= temp)
                break;
            a[i] = a[j];
            i = j;
```

```

    }
    a[i] = temp;
}

```

- 2、堆的另外一个常用场合是求 top k 问题。假设我们已经有一个 n 个元素的最小堆，如果用最直观的方法求最小的 k 个元素，其计算复杂度为  $O(k\log n)$ 。试问：如何在更小的复杂度内求 top k 问题？用伪代码描述你的方法，并给出复杂度证明。(7 分)

**答：**

假设原最小堆为 H，新建一个最小堆 H'，H'初始化为空。

将最小堆 H 的堆顶元素 a 插入 H'；然后以下步骤执行 k 次：

将 H'的堆顶元素 a'删除并输出，然后将 a'在 H 中的两个孩子插入 H'中。

为了快速找到 a'在 H 中的两个孩子，H'存储的实际上是元素在 H 中的下标，而大小比较通过下标读取 H 中对应的元素。

由于 k 次对 H'的删除操作，最多将 2 个新元素插入 H'中，因此 H'的大小不超过 2k。每次插入删除操作复杂度都为  $O(\log 2k)$ ，因此算法复杂度为  $O(k\log k)$ 。

**评分标准：满分：** $O(k\log k)$ 或者更好。若提出了快速排序的 partition 函数 ( $O(n)$  复杂度)，给 4 分。

- 3、给定一个无向图  $G(V, E)$ ，V 为顶点集合，E 为边集合。对于 V 中的两个顶点 u 和 v，如果图中存在一条 u 到 v 的路径，则称 u 和 v 彼此可达。设计一种数据结构，将图 G 进行预处理得到该数据结构，使得对于任意 u 和 v，通过该数据结构可以在常数时间得到 u 和 v 是否彼此可达。要求该数据结构存储开销为  $O(|V|)$ ，预处理复杂度为  $O(|V| + |E|)$ 。(8 分)

注意——图中的结点从 0 开始用整数连续编号。图 Graph 上的操作有：取得结点总数 `int GetVertexNum()`；取得 u 之后的 v 的下一个邻接顶点 `int GetNextNeighbor(int v,int u)`，u 为-1 则取得 v 的第一个邻接顶点，返回-1 表示没有下一个邻接顶点了；可以直接使用数组、链表、栈和队列这些基本数据结构及其上的基本操作，但要注释每个所使用操作的含义。

- (1) 描述所设计的数据结构，描述如何得到该数据结构，并写出预处理的代码。
- (2) 描述如何通过该数据结构在常数时间内得到 u 和 v 是否可达，并写出相应代码。

**答：** 本题考察学生对连通分量的掌握和运用。使用 BFS 算法得到图的 N 个连通分量，为每个连通分量分配一个 id。用一个包含  $|V|$  个元素的数组 `connectedCom[ $|V|$ ]`，记录每个顶点属于的连通分量 id，即 `connectedCom[u]` 表示 u 属于的联通分量。对于 u 和 v，若 `connectedCom[u] = connectedCom[v]`，则 u 和 v 彼此可达。

**评分标准：** 本题答案不唯一，可以是 BFS、DFS，也可以是并查集。

总共 8 分。

思路 2 分——思路清晰正确为 2 分，方法不限于参考答案这一种，思路不完全正

确最多为 1 分，没写为 0 分；算法 1-6 分。

## ds期中

### 填空

- 1 后缀转中缀表达式
- 2 循环队列的元素个数
- 3 对称矩阵的存储，给首地址，再给个地址，问存的是谁
- 4 广义表的表头和表尾

### 选择

- 1 大O，问题规模依次减半  $\lg n$
- 2 带头节点的单循环链表在释放头结点后一个结点时的操作
- 3 静态链表
- 4 共享栈（一头一尾的那个）
- 5 kmp的nextValue数组（失配情况下往右滑动多少）
- 6 对称矩阵的存储，把下三角按行优先存，已知a00地址100，a33是220，求a55
- 7 有向图的度
- 8 给定一张图里边的最大值，告诉不联通，问结点个数（完全图+鼓励点）
- 9 二叉树的顺序存储（数组）和链式存储（node和指针）的比较，给定特定条件，什么情况下数组更优？
- 10 已知完全二叉树第七层有10个叶子结点，问总结点可能个数
- 11 Huffman树已有两个字符编码为0，10，码长最多4，最多还能编几个字符？
- 12 给一棵二叉树表示的森林的前序和中序，还原森林，问森林里有几棵树
- 13 已知图有1k点和2k条边，一个点要2B，一条边要3B，以下四个数种那个最接近存储规模？
- 14 以下关于并查集的说法哪个是错误的

### 解答

- 1 给森林的先根和后根遍历的序列，还原二叉树表示（左孩子右兄弟）；把第一问的树后续线索化
- 2 给定邻接矩阵 分别用Prim和Kruskal求最小生成树
- 3 dij

### coding

- 1 leetcode permutation（求1到n的全排列）
- 2 build a Huffman Tree
- 3 给邻接表类定义，并在邻接表的基础上做bfs（可能不联通）

复旦大学计算机科学技术学院

2022~2023 学年第 1 学期期末考试试卷

☒A 卷      ☐B 卷      ☐C 卷

课程名称: 数据结构 课程代码: COMP130004.02, 03  
开课院系: 计算机科学技术学院 考试形式: 闭卷  
姓 名: \_\_\_\_\_ 学 号: \_\_\_\_\_ 专 业: \_\_\_\_\_

提示: 请同学们秉持诚实守信宗旨, 谨守考试纪律, 摒弃考试作弊。学生如有违反学校考试纪律的行为, 学校将按《复旦大学学生纪律处分条例》规定予以严肃处理。

|     |   |   |   |   |     |
|-----|---|---|---|---|-----|
| 题 号 | 一 | 二 | 三 | 四 | 总 分 |
| 得 分 |   |   |   |   |     |

一、填空题 (20 分, 每题 2 分)

1、 给出以下算法的渐进时间复杂度为\_\_\_\_\_。

```
long fun(int n) {  
    if (n <= 1) {  
        return 1;  
    } else {  
        return fun(n-1) + fun(n-2);  
    }  
}
```

2、 按教材中循环队列的实现方式, 设循环队列的容量为 50 (序号从 0 到 49), 现经过一系列的入队和出队运算后, 有 front=16, rear=5 (rear 指向队尾元素的后一位置), 则当前循环队列中元素个数为\_\_\_\_\_。

3、 设一个栈的入栈序列为 1, 2, 3, ..., n, 其出栈序列为 P<sub>1</sub>, P<sub>2</sub>, ..., P<sub>n</sub>。若 P<sub>n</sub> 的值是 1, 则 P<sub>i</sub> 的值是\_\_\_\_\_。

4、 规定根结点的层次为 1, 已知完全二叉树的第 7 层有 10 个叶子结点, 则整棵完全二叉树的结点数最多为\_\_\_\_\_。

5、 对 n 个互不相同的符号进行霍夫曼编码, 若生成的霍夫曼树共有 115 个结点, 则 n 的值是\_\_\_\_\_。

6、 给定一个有序线性表 {4, 7, 9, 10, 13, 15, 16, 18, 20, 23, 26, 30}, 利用折半搜索算法搜索关键字为 23 的元素时, 需要\_\_\_\_\_次比较即可搜索到最终相匹配的元素。

7、 把数字 20, 22, 61, 43, 81, 34, 100, 2023, 12 依次插入一个空的平衡二叉树 (AVL Tree, 中序为升序), 第一次进行 RL 旋转是在插入\_\_\_\_\_后。

- 8、 在具有  $n$  个顶点的有向图中，每个顶点的度最大可达\_\_\_\_\_。
- 9、 在 AOE 网络中，若任意一个非关键活动延期完成，则整个工程\_\_\_\_\_不会延期完成；若任意一个关键活动提前完成，则整个工程\_\_\_\_\_会提前完成。（可以填“一定”或者“不一定”）
- 10、在冒泡排序、插入排序、选择排序、快速排序、归并排序、堆排序、以及基数排序中，最坏情况和最好情况下时间复杂度相同的排序算法有\_\_\_\_\_。

## 二、选择题（10 分，每题 2 分）

- 1、 在单链表中，若  $p$  所指的结点不是最后结点，在  $p$  之后插入  $s$  所指结点，则执行\_\_\_\_\_。
- A  $p \rightarrow \text{link} = s; \quad s \rightarrow \text{link} = p;$                       B  $s \rightarrow \text{link} = p \rightarrow \text{link}; \quad p \rightarrow \text{link} = s;$   
C  $s \rightarrow \text{link} = p \rightarrow \text{link}; \quad p = s;$                       D  $s \rightarrow \text{link} = p; \quad p \rightarrow \text{link} = s;$
- 2、 广义表  $LS = (a, b, (c, d), (e, (f, g)))$ ，假设求表头操作为  $\text{Head}$ ，求表尾部操作为  $\text{Tail}$ ，则  $\text{Head}(\text{Tail}(\text{Head}(\text{Tail}(\text{Tail}(LS))))$  的值为\_\_\_\_\_。
- A (g)                      B (d)                      C c                      D d
- 3、 某二叉树前序遍历序列为 E, F, H, I, G, J, K，中序遍历序列为 H, F, I, E, J, K, G，则该二叉树根结点的右子树的根结点为\_\_\_\_\_。
- A E                      B F                      C G                      D H
- 4、 下面关于求关键路径的说法不正确的是\_\_\_\_\_。
- A 求关键路径是以拓扑排序为基础                      B 关键路径可能不止一条  
C 关键路径是从源点到终点的最短路径                      D 关键活动一定位于关键路径上
- 5、 要设计一个程序，根据输入的一个英文文本文件，统计其中所有英文单词出现的频率，这里的英文单词就是指英文字典中的单词。在以下的数据结构中，能实现时间效率最高的统计的数据结构为\_\_\_\_\_。
- A 二叉搜索树                      B 散列表                      C 顺序表                      D 堆

## 三、问答题（30 分，每题 6 分）

- 1、 下面是判断一颗二叉树是否为二叉搜索树（左孩子结点关键码小于双亲结点关键码，双亲结点关键码小于右孩子结点关键码）的算法，请补充完整（假设第一次调用  $\text{is\_bst}$  时知道树上的最大值和最小值）。（6 分）

```

struct node {
    int left_child, right_child; //存储当前结点的左右孩子编号，若不存在则为 0
    int value;
} tr[10000];

bool is_bst(int root, int lower_bound, int upper_bound) {
    if (root == 0)
        return ____ (1) ____;

    int cur = tr[root].value;

    if ((cur > lower_bound) && (cur < ____ (2) ____))
        && is_bst(tr[root].left_child, lower_bound, ____ (3) ____))
        && is_bst(____ (4) ____, ____ (5) ____, ____ (6) ____))

    return true;

    return false;
}

```

- 2、 若以关键码序列 {Thu, Tue, Wed, Last, Fri, Sat, Mon, Sun, Next} 作为输入序列。(6 分)
  - (1) 试画出从一棵空树开始，依上述顺序（从左到右）输入关键码，用 AVL 树的查找与插入算法生成一棵 AVL 树的过程，并说明生成过程中采用何种旋转方式进行平衡调整。
  - (2) 若 AVL 树中有  $n$  个结点，其高度为  $h$ ，请指出在最坏情况下，对该树进行插入一个结点、删除一个结点与依次输出全部结点这三个操作各自的渐进时间复杂度。
- 3、 举例说明当存在负权值边时，即使没有负权值回路，Dijkstra 也不能正确找到最短路径。(6 分)
- 4、 已知一组关键码为 {26, 36, 41, 38, 44, 15, 68, 12, 51, 25, 06}，假设装填因子  $\alpha=0.75$ ，用开散列法解决冲突。(6 分)
  - (1) 试用除留余数法构造出哈希函数，并画出哈希表。
  - (2) 请计算出等概率情况下查找成功与失败的平均搜索长度。

5、请分析并回答以下问题：（6分）

- (1) 设要求从大到小排序，在什么情况下冒泡排序算法关键码交换的次数为最大？
- (2) 已知某文件的记录关键码集为{50, 10, 50, 40, 45, 85, 80}，请选择一种从平均性能而言是最佳的基于比较的内排序方法进行排序，并给出每一趟排序后的结果。

#### 四、算法题（40分）

（除写出相应算法之外，还需要说明程序和数据结构的设计思路）

- 1、如果按照关键码的输入顺序从空树开始建立一棵二叉搜索树，不同的输入顺序可能产生相同的二叉搜索树。例如，输入序列{3, 2, 4, 1}与{3, 4, 2, 1}生成的二叉搜索树是相同的，但与序列{3, 1, 2, 4}生成的二叉搜索树不相同。假设序列中的关键码均不相等，给定两个关键码输入序列，请设计时空效率尽量优的相关数据结构和算法判断它们生成的二叉搜索树是否相同，并分析算法时间复杂度。（10分）
- 2、给出有向图的邻接表类定义，并定义其上的非递归的深度优先搜索方法。类定义可适当简化，可省略与所定义的深度优先搜索方法无关的属性和方法的定义。（15分）
- 3、给定高度 $m$ 、宽度 $n$ 的一张 $m*n$ 的乘法表、以及正整数 $k$ ， $k$ 的范围在 $[1, m*n]$ 之间，请用时空效率尽量优的程序返回表中第 $k$ 小数字的值。（15分）

例 1:

输入:  $m = 3, n = 3, k = 5$

输出: 3

解释:

乘法表:

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 4 | 6 |
| 3 | 6 | 9 |

第5小的数字是3 (1, 2, 2, 3, 3)

例 2:

输入:  $m = 2, n = 3, k = 6$

输出: 6

解释:

乘法表:

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 2 | 4 | 6 |

第6小的数字是6 (1, 2, 2, 3, 4, 6)

# 期中考试讲解

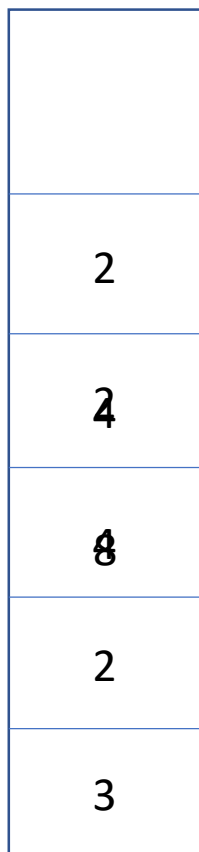
# 填空题

# 填空题

- 1、算法“`int i=0, s=0; while (s<=n) { i++; s=s+i; }`”的时间复杂度为 $O(\underline{\hspace{2cm}})$ 。
- 答：由于S变化是 $1+2+3+\dots+k = k(k+1)/2$   
而i变化是 $1+1+1+\dots = k$   
s变化比i快，所以用s会提前终止循环。  
循环次数是 $k(k+1)/2=n$   
解得复杂度是 $O(\sqrt{n})$

- 2、表达式  $3*2^{(4+2*2-6*3)}-5$  求值过程中当检查到6时，数据栈栈顶的数据值为\_\_\_\_\_。
- 解：首先把表达式转化成逆波兰后缀表达式

$3\ 2\ 4\ 2\ 2\ * \ +\ 6\ 3\ * \ - \ ^ \ * \ 5 \ -$



\*

+

- 3、单循环链表表示的队列长度为 $n$ ，若只设头指针，则进队的时间复杂度为\_\_\_\_\_。
- 答：只设头指针

因为进队是从队尾进入，而只知道头指针，想找到尾指针的时间复杂度是 $O(n)$ ，找到之后插入操作复杂度是 $O(1)$ 所以进队复杂度是 $O(n)$

- 4、已知广义表 $LS=(a, (b, c, d), e)$ ，假设求表头操作作为 $Head$ ，求表尾操作作为 $Tail$ ，则 $b=$ \_\_\_\_\_。
- 答：广义表第一个元素为表头，其余元素组成的表为表尾。
- 1)  $tail(LS)=((b,c,d),e)$
- 2)  $tail(tail(LS)) = (b,c,d)$
- 3)  $head(tail(tail(LS)))=b$

- 5、已知三对角矩阵 $A[1..9, 1..9]$ 的每个元素占用2个单元，现将其三条对角线上的元素逐行存储在起始地址为1000的连续内存单元中，则元素 $A[7][8]$ 的存储地址为\_\_\_\_\_；如按列优先顺序进行存储，则 $A[7][8]$ 的存储地址为\_\_\_\_\_。
- 答：由于矩阵较小，方法一可以用数的办法。
- 对于本题从1开始计数的三对角矩阵，共有 $2 + (i-2) * 3 + j - i + 2 = 2 * i + j - 2$ 个数据需要存储，存储地址为 $2 * i + j - 3$
- $A[7][8] = 1000 + 2 * 19 = 1038$
- $A[8][7] = 1000 + 2 * 20 = 1040$

- 6、已知完全二叉树的第7层有10个叶子节点，则整棵二叉树的结点数最多是\_\_\_\_\_。

- 答：根节点为0层

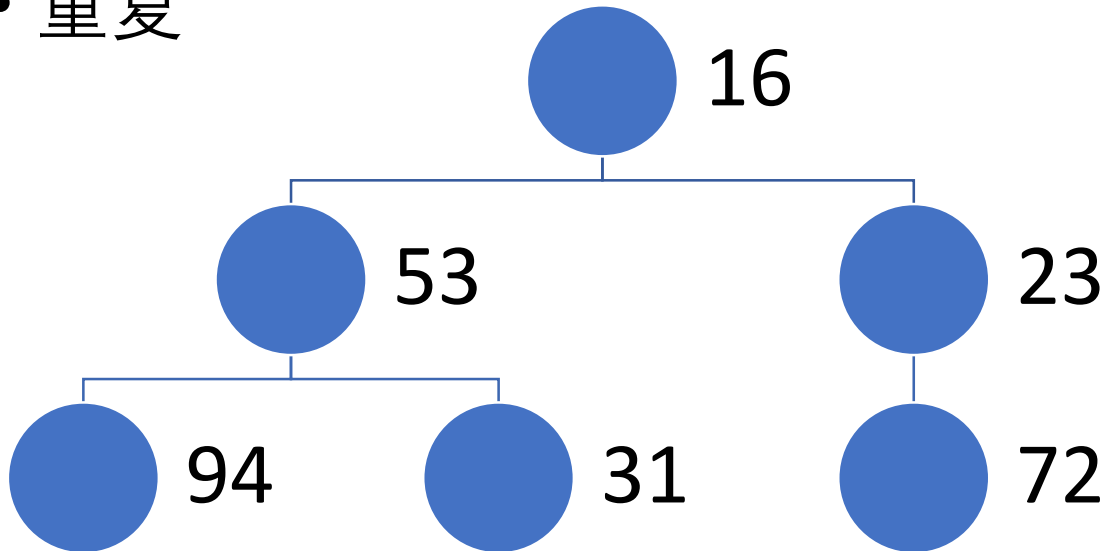
节点数最多的情况是，第七层全满，第八层最后缺少20个结点。

$$2^{7-0+1} + 2^8 - 1 - 20 = 491$$

- 7、一棵二叉树的中序遍历序列为 $ABCDEFGH$ ，后序遍历序列为 $GFEDCBA$ ，则这棵二叉树根结点左子树中的结点数目为\_\_\_\_\_。
- 答：后序遍历最后一个为根节点，所以根节点是A，中序遍历根节点将树分为左右两个子树。  
 $ABCDEFGH$ ，所以可以看出根节点左子树节点数目为0

- 8、将一个有 $n$ 个结点的森林用左孩子-右兄弟链表示时，其中空链域的个数为\_\_\_\_\_。
- 答： $n+1$ ，只跟二叉树有关，跟什么二叉树没有关系

- 9、将关键码序列{16, 53, 23, 94, 31, 72}转换为最大堆后所对应的关键码序列为\_\_\_\_\_。
- 答：从  $(n/2)$  处开始进行判断，如不满足进行交换，交换下去的结点递归判断是否满足最大堆，如都满足则， $n=n-1$ .
- 重复



# 算法填空

# 算法填空题

- restore函数用于重构树
- postorder函数用于生成后序遍历序列
- 答案：

- 1) \*ppos
- 2) rpos=ipos
- 3) rpos-ipos
- 4) ipos
- 5) ppos+1

```
TNODE *restore(char *ppos, char *ipos, int n) {  
    TNODE *ptr;  
    char *rpos; int k;  
    if (n<=0) return NULL;  
    ptr->info=(1)_____;  
    for ((2)_____; rpos<ipos+n; rpos++)  
        if (*rpos==*ppos) break;  
    k=(3)_____;  
    ptr->llink=restore(ppos+1, (4)_____, k);  
    ptr->rlink=restore ((5)_____+k, rpos+1, n-1-k);  
    return ptr;  
}
```

# 问答题

# 问答题2

- 设目标串 $T = \text{"xyxxxxxyxyx"}$ ，模式串 $P = \text{"xyxy"}$ 。如何用最少的比较次数找到 $P$ 在 $T$ 中第一次出现的位置？相应的比较次数是多少？请给出具体的分析过程。  
(6分)

- 答：首先写出next数组 $[-1, 0, 1, 0, 1]$

- $xxyxxxxxyxyx$

- $xxyx\color{red}y$   $\color{red}5$

- $\quad\quad xx\color{red}y$   $\color{red}2$

- $\quad\quad\quad xxyx\color{red}y$   $\color{red}4$

- $\quad\quad\quad\quad xx\color{red}y$   $\color{red}2$

- $\quad\quad\quad\quad\quad xx\color{red}y$   $\color{red}2$

- $\quad\quad\quad\quad\quad\quad xxyxy$   $4$

# 标准答案

设目标串  $T = \text{"xxvxxxvxxxvxxv"}$ ，模式串  $P = \text{"xxvxy"}$ 。如何用最少的比较次数找到  $P$  在  $T$  中第一次出现的位置？相应的比较次数是多少？请给出具体的分析过程。（6分）。

答：穷举模式匹配算法的时间复杂度为  $O(m*n)$ （其中， $m$  为模式串长度， $n$  为目标串长度）。KMP 算法有一定改进，时间复杂度达到  $O(m+n)$ 。本题也可采用从后面匹配的方法，即从右向左扫描，比较 6 次成功。另一种匹配方式是从左往右扫描，但是先比较模式串的最后一个字符，若不等，则模式串后移；若相等，再比较模式串的第二个字符，若第一个字符也相等，则从模式串的第二个字符开始向右比较，直至相等或者失败。若失败，模式串后移，再重复以上过程。按这种方法，本题比较 18 次成功。

# 算法二

Total = 18次

|   |   |    |   |    |    |    |    |    |    |    |    |    |    |   |
|---|---|----|---|----|----|----|----|----|----|----|----|----|----|---|
| x | x | y  | x | x  | x  | y  | x  | x  | x  | x  | y  | x  | y  | x |
|   |   |    |   | bf |    |    |    |    |    |    |    |    |    |   |
|   |   |    |   |    | bf |    |    |    |    |    |    |    |    |   |
|   |   | bf |   |    |    | bt |    |    |    |    |    |    |    |   |
|   |   |    |   |    |    |    | bf |    |    |    |    |    |    |   |
|   |   |    |   |    |    |    |    | bf |    |    |    |    |    |   |
|   |   |    |   |    |    |    |    |    | bf |    |    |    |    |   |
|   |   |    |   |    |    |    |    |    |    | bf |    |    |    |   |
|   |   |    |   |    |    |    | b  | b  | bf |    | bt |    |    |   |
|   |   |    |   |    |    |    |    |    |    |    |    | bf |    |   |
|   |   |    |   |    |    |    |    |    | b  | b  | bt | bf | bt |   |

# 问答题1

某算法时间复杂度的递推关系如下，其中  $n$  为求解问题的规模，设  $n$  是 3 的正整数幂。请给出该算法时间复杂度的大  $O$  表示法及推导过程。（6 分）

出错的同学较多

2、最后结果错误

解题方法：

递归树法

\*主方法

$$T(n) = \begin{cases} 1 & (n = 1) \\ 5T(n/3) + n & (n > 1) \end{cases}$$

答：  $T(n) = 5T(n/3) + n = 5(5T(n/9) + n/3) + n = \dots$

$$= 5^{\log_3 n} + 5^{\log_3 n - 1} * 3 + 5^{\log_3 n - 2} * 3^2 + \dots + 5 * n/3 + n$$

$$T(n) * 5/3 = 5/3 * 5^{\log_3 n} + 5^{\log_3 n} + 5^{\log_3 n - 1} * 3 + \dots + 5 * n/3$$

$$T(n) * 5/3 - T(n) = 5/3 * 5^{\log_3 n} - n$$

$$T(n) = (3/2)(5^{\log_3 n + 1}/3 - n) = O(5^{\log_3 n} - n)$$



# 问答题3

利用  $n$  个从左到右顺序排列的容量足够大的栈，初始时有  $2^n$  个不同元素位于这  $n$  个栈的左侧。限定位于  $n$  个栈左侧的  $2^n$  个不同元素是按顺序进入  $n$  个栈，从  $n$  个栈出来后则在右边按出栈顺序排列。证明通过一系列的进栈和出栈操作，要求操作的方向从左向右，最终在这  $n$  个栈的右边可以形成这  $2^n$  个元素的  $2^n!$  种不同的元素排列顺序。（提示：利用数学归纳法）（6 分）

本题问题不大。90%的同学都做对了，部分同学被扣分，是因为没有描述清楚，部分同学留白。

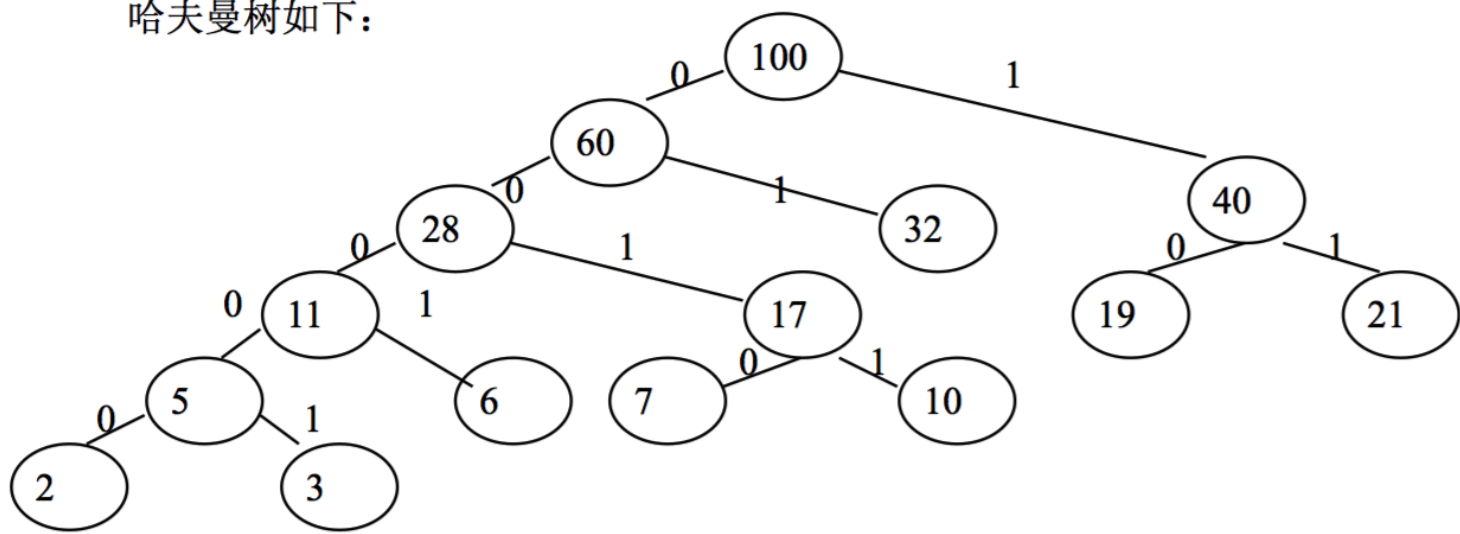
- 1、只要证明产生的输出序列是输入序列中的元素的任意排列组合即可。
- 2、 $n = 1$  时显然成立
- 3、假设  $n = k + 1$  时，元素有  $2 * 2^k$  个，先把前面  $2^k$  个入栈，这时候可以先让第一个栈中的元素通过剩下的  $n$  个栈，共有  $2^k!$  种不同的元素排列顺序。
- 4、也可以先让不在第一个栈的元素通过剩下的  $n$  个栈，也有  $2^k!$  种不同的元素排列顺序。
- 5、这两个  $2^k!$  种不同的排列相互之间可前可后也可以交错形成任意顺序，所有共有  $(2 * 2^k)! = 2^{k+1}!$  种不同的排列。

# 问答题4

用于通信的电文由 8 个字符组成，字符在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10。试为这 8 个字符设计哈夫曼编码。使用 0~7 的 3 位等长二进制表示形式是另一种编码方案，对于上述实例比较两种方案的优缺点。（6 分）

答：哈夫曼编码方案的带权路径长度为 2.61，而用 0~7 的 3 位等长二进制表示形式的路径长度为 3。显然前者降低了信息编码的平均长度，可以提高信道利用率，提高信息传输速度，节省存储空间。两种编码的对照表如下（字符的出现频率转换成整数）：

哈夫曼树如下：



|     |      |     |       |      |     |       |     |     |
|-----|------|-----|-------|------|-----|-------|-----|-----|
| 频数  | 7    | 19  | 2     | 6    | 32  | 3     | 21  | 10  |
| 哈夫曼 | 0010 | 10  | 00000 | 0001 | 01  | 00001 | 11  | 011 |
| 等长码 | 000  | 001 | 010   | 011  | 100 | 101   | 110 | 111 |

# 哈夫曼 缺点

- 第一，每一个树的节点都要储存有关它们的父节点与子节点等等相关资讯，如果符号集合的数量包含许多不同机率的符号，记忆体的负荷量会明显增大许多。
- 第二，哈夫曼树的追踪需要耗费极大的运算量。
- 所以基于以上两个论点，传统的哈夫曼编码是一种极为消费储存空间且没有效率的方式。

# 问答题5

已知指向二叉树中某结点的指针  $p$ ，并且已知该二叉树为中序线索化二叉树，规定二叉树的中序第一个结点的左孩子指针为空，最后一个结点的右孩子指针为空。请问仅根据这些已知条件什么情况下能找到  $p$  的双亲结点，什么情况下无法找到？（6分）

- 主要存在的问题

- 20%左右的同学没做
- 大部分同学作答不全(两条路)

1、先检查 $p$ 的左孩子的指针，再检查左孩子的左孩子，一直到遇到线索，如果线索指针为空，说明到底第一个结点，则需要从 $p$ 的右孩子找。

2、如果线索非空，则到达 $p$ 的一个祖先结点 $q$ ，先找到 $q$ 的右孩子 $r$ ，检查 $r$ 是否是 $p$ ，如果是的话则 $q$ 为 $p$ 的双亲，否则沿着 $r$ 的左孩子一路找到 $p$ ，在 $p$ 之前找到的就是 $p$ 的双亲。

3、从 $p$ 的右孩子找也类似。如果 $p$ 是树根，从 $p$ 的左孩子找到第一个结点，从 $p$ 的右孩子找到最后一个结点，则无法找到 $p$ 的双亲。

4、因为从根结点沿左子女链一定走到中序序列的第一个结点，沿右子女链一定走到中序序列的最后一个结点，根结点无父节点，因此无法找到。

# 算法设计题

# 算法设计1

- 一个数组中存放着未经排序的一组正整数，设计算法调整数组中的数据顺序，使得所有奇数存放在所有偶数之前。要求尽量少的辅助空间和尽量快的速度。

要求:(1)描述算法的基本思想并分析时间和空间复杂度;(2)给出程序代码。(10 分)

答:(1)一遍扫描，用类似于快排的交流过程。时间复杂度  $O(n)$ ，空间复杂度  $O(1)$

# 算法设计1

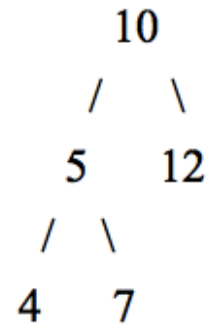
```
void move(int A[ ], int n) {  
    int i=0, j=n-1, tmp;  
    while (i<j) {  
        while (i<j&&A[i]%2==1) i++;  
        while(i<j&&A[j]%2==0) j--;  
        if(i<j) {  
            tmp=A[i]; A[i]=A[j]; A[j]=tmp;  
            i++; j--;  
        }  
    }  
} //算法结束
```

# 算法设计2

- 输入一个整数和一棵二叉树。从树的根结点开始往下访问一直到叶结点所经过的所有结点形成一条路径。打印出和与输入整数相等的所有路径。例如，输入整数 22 和如下二叉树：

则打印出两条路径:10, 12 和 10, 5, 7。二叉树结点的数据结构定义为:

```
struct BinaryTreeNode {  
    int m_nValue; //value of node  
    BinaryTreeNode *m_pLeft; //left child of node  
    BinaryTreeNode *m_pRight; //right child of node  
};
```



要求:(1) 描述该算法的基本设计思想;(2) 给出程序代码。(15 分)

# 算法设计2

- 主要利用回溯和递归，需要一个栈辅助路径的回溯。

```
struct TreeNode {  
    int data;  
    TreeNode *left;  
    TreeNode *right;  
};  
void printPaths(TreeNode  
*root, int sum) {  
    int path[MAX_HEIGHT];  
    helper(root, sum, path, 0);  
}
```

# 算法设计2

```
void helper(TreeNode * root, int sum, int path[ ], int top) {  
    path[top++]=root.data;  
    sum-=root.data;  
    if(sum>=0) { //剪枝  
        if (root->left==NULL && root->right==NULL)  
            if (sum==0) printPath(path, top);  
        else {  
            if (root->left!=NULL)  
                helper(root->left, sum, path, top);  
            if (root->right!=NULL)  
                helper(root->right, sum, path, top);  
        }  
    }
```

# 算法设计题3

- 在二叉树中查找值为  $x$  的结点，试编写算法打印值为  $x$  的结点的所有祖先(假设值为  $x$  的结点不多于一个)，并分析算法的渐进时间复杂度(不加分析直接写出结果得零分)。(15 分)

程序设计思想——可以利用栈来解答本题。通过前序遍历树，把路经的结点都放入栈中，如果遇到结点  $x$  就输出栈里面的元素;否则，利用前序遍历，递归处理其它结点。

# 算法设计题3

```
class BTree {  
    private:  
        BTreeNode *root; //树的根结点  
    public:  
        void ancestor(BTreeNode *current);  
        stack S;  
};
```

# 算法设计题3

```
class BTree {  
    private:  
        BTreeNode *root; //树的根结点  
    public:  
        void ancestor(BTreeNode *current);  
        stack S;  
};
```

# 算法设计题3

```
void BTree :: ancestor(BTreeNode * current) {  
    if(current==NULL) return false;  
    S.push(root);  
    if (root->left==x)  
        { 把栈中的元素依次退出栈,  
          作为结果输出;  
        } else {  
            ancestor (current->left);  
            ancestor (current->right);  
        }  
}
```

复旦大学计算机科学技术学院

《数据结构》期末考试试卷

A 卷 共 7 页

课程代码：INFO130051.02-03      考试形式：□开卷    ☒闭卷      2010 年 1 月  
(本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效)

专业\_\_\_\_\_ 学号\_\_\_\_\_ 姓名\_\_\_\_\_ 成绩\_\_\_\_\_

|    |   |   |   |   |    |
|----|---|---|---|---|----|
| 题号 | 一 | 二 | 三 | 四 | 总分 |
| 得分 |   |   |   |   |    |

一、填空题（20%）

1、 以下算法的时间复杂度为\_\_\_\_\_。

```
void function(int n)

{

    int i=1; s = 1;

    while(s<n) s += ++i;

    return i;

}
```

2、 对给定的 n 个元素，建立一个有序单链表的时间复杂度是\_\_\_\_\_。

3、 高度为 i (i ≥1) 的完全二叉树最多有\_\_\_\_\_个结点，最少有\_\_\_\_\_个结点。若按从上到下，从左到右顺序从 1 开始给结点编号，则编号最小的叶子结点的编号为\_\_\_\_\_。

4、 有 16 条边的无向图，有 3 个 4 度顶点，有 4 个 3 度顶点，其余顶点的度数都小于 3，则此图至少有\_\_\_\_\_个顶点。

5、 n 个结点和 e 条边构成的无向图的邻接矩阵中，零元素的个数为\_\_\_\_\_。

6、  $n$  个顶点  $e$  条边的图的深度优先遍历的空间复杂度为\_\_\_\_\_。

7、 由 25、10、60、50 构成的二叉排序树有\_\_\_\_\_种；其中 AVL 树有\_\_\_\_\_种。

## 二、选择题（10%）

1、 对于顺序查找，假定查找成功与不成功的概率相同，对每个记录查找的概率也相同，则顺序查找的平均查找长度为\_\_\_\_\_。

- |   |            |   |              |
|---|------------|---|--------------|
| A | $0.5(n+1)$ | B | $0.25(n+1)$  |
| C | $0.5(n-1)$ | D | $0.75n+0.25$ |

2、 下列 算法占用的辅助空间最多。

- A 堆排序                      B 希尔排序                      C 快速排序                      D 归并排序

3、 设有无向图  $G = (V, E)$  和  $G' = (V', E')$ ，如  $G'$  是  $G$  的生成树，则下面错误的说法为\_\_\_\_\_。

- A  $G'$  是  $G$  的连通分量  
B  $G'$  是  $G$  的无环子图  
C  $G'$  是  $G$  的子图  
D  $G'$  是  $G$  的极小连通子图且  $V' = V$

4、 二叉树在线索化后, 仍不能有效求解的问题是\_\_\_\_\_。

- A 前序线索化二叉树中求前序后继      B 中序线索化二叉树中求中序后继
- C 中序线索化二叉树中求中序前驱      D 后序线索化二叉树中求后序后继

5、 下列叙述中不正确的是\_\_\_\_\_。

- A 任何一个无向连通图有一棵或者多棵最小生成树。
- B 克鲁斯卡尔算法比较适合边稠密的图，普里姆算法比较适合边稀疏的图。
- C 克鲁斯卡尔算法与普里姆算法均可能生成不同的最小生成树。
- D 有向完全图一定是强连通有向图。

### 三、问答题 (35%)

- 1、已知一个有向网络的邻接矩阵 A 如下，现在要在该网络中的某个结点建立超市，要求该结点距离其它各结点的最长往返路程最短，在最长往返路程相等的情况下，则选择到其它所有结点的往返路程总和最短的结点。问应该选择哪个结点，给出解题过程（结点编号为 1-6）。(10 分)

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | - | 0 | - | 4 | - |
| 1 | - | - | 0 | 1 | - |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

- 2、在 AVL 树中插入一个新结点后，怎么判断需要调整 AVL 树恢复平衡？怎样判断这种调整可以结束，不再需要进一步的调整？(5 分)

3、在  $n$  个不同元素的顺序表中找出最大和最小值需要至少进行多少次比较？算法的思路是怎样的？（6 分）

4、对数据元素 1、13、29、18、75、60、43、54、90、46、21、34，采用散列函数  $\text{hash}(\text{key}) = \text{key} \% 13$  和线性探测再散列方法构造散列表 HT[13]。（1）请计算出各元素散列地址。（2）求搜索成功和不成功的平均搜索长度。（6 分）

- 5、若存于顺序存储结构中的  $n$  个元素的序列近似有序，即去掉其中少数  $k$  个元素的序列是有序序列，试对这种数据文件讨论用直接插入排序法、起泡排序法和直接选择排序法的渐进时间复杂度。(8 分)

#### 四、算法题 (35%)

(除写出相应算法之外，还需要说明算法和数据结构的设计思路，算法用 c 或者 C++ 语法描述)

- 1、设  $T$  是一棵满二叉树，写一个递归算法，把  $T$  的前序遍历序列转换成后序遍历序列。(10 分)

- 2、设  $A$ 、 $B$  是长度为  $n$  的两个非降序数组。如果将这  $2n$  个数全体排序，处于位置  $n$  的数称为中位数。设计一个最坏情况下  $O(\log n)$  的算法，求这个中位数。并且证明你的算法的时间复杂性。  
(15 分)

- 3、设计一个算法，判断一个有向无环图中是否存在这样的顶点，该顶点到其它任意顶点都有一条有向路。（10 分）

（装订线内不要答题）

# 复旦大学计算机科学技术学院

## 《数据结构》期末考试答案及评分标准

### A 卷 共 7 页

课程代码: INFO130051.02-03 考试形式: ☐ 开卷 ☒ 闭卷

2010 年 1 月

(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

#### 一、填空题 (20%)

- 1、 答:  $O(n^{1/2})$
- 2、 答:  $O(n^2)$
- 3、 答:  $2^{i-1}, 2^{(i-1)}, 2^{(i-2)+1}$
- 4、 答: 11
- 5、 答:  $n^2 - e$
- 6、 答:  $O(n)$
- 7、 答: 14 5 (15? 4?)

#### 二、选择题 (10%)

- 1、 答: d
- 2、 答: d
- 3、 答: a
- 4、 答: d
- 5、 答: b

#### 三、问答题 (35%)

- 1、 已知一个有向网络的邻接矩阵 A 如下, 现在要在该网络中的某个结点建立超市, 要求该结点距离其它各结点的最长往返路程最短, 在最长往返路程相等的情况下, 则选择到其它所有结

点的往返路程总和最短的结点。问应该选择哪个结点，给出解题过程（结点编号为 1-6）。（10 分）

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | - | 0 | - | 4 | - |
| 1 | - | - | 0 | 1 | - |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

答：先用 floyd 算法算出所有结点之间的最短路径，然后对每个结点 i 算出到其它各结点的往返距离总和  $\text{sum}(i)$ ，以及 i 到其它结点的最长距离  $\text{long}(i)$ ，最后选择结点 5。

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | - | - | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | 6 | 0 | - | 4 | 7 |
| 1 | 3 | - | 0 | 1 | 4 |
| - | 1 | - | - | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | - | 3 |
| - | 0 | 3 | 2 | - | - |
| 4 | 6 | 0 | 8 | 4 | 7 |
| 1 | 3 | 6 | 0 | 1 | 4 |
| - | 1 | 4 | 3 | 0 | 3 |
| - | - | 2 | 5 | - | 0 |

|   |   |   |   |   |    |
|---|---|---|---|---|----|
| 0 | 2 | 5 | 4 | 9 | 3  |
| 7 | 0 | 3 | 2 | 7 | 10 |
| 4 | 6 | 0 | 8 | 4 | 7  |
| 1 | 3 | 6 | 0 | 1 | 4  |
| 8 | 1 | 4 | 3 | 0 | 3  |
| 6 | 8 | 2 | 5 | 6 | 0  |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 6 | 0 | 8 | 4 | 7 |
| 1 | 3 | 6 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 8 | 2 | 5 | 6 | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 5 | 0 | 7 | 4 | 7 |
| 1 | 2 | 5 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 7 | 2 | 5 | 6 | 0 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 4 | 5 | 3 |
| 3 | 0 | 3 | 2 | 3 | 6 |
| 4 | 5 | 0 | 7 | 4 | 7 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 5 | 0 | 1 | 4 |
| 4 | 1 | 4 | 3 | 0 | 3 |
| 6 | 7 | 2 | 5 | 6 | 0 |

Sum(1)=2+5+4+5+3+3+4+1+4+6=37

Sum(2)=3+3+2+3+6+2+5+2+1+7=34

Sum(3)=4+5+7+4+7+5+3+5+4+2=46

Sum(4)=4+2+7+3+5+1+2+5+1+4=34

Sum(5)=5+3+4+1+0+6+4+1+4+3+0+3=34

Sum(6)=3+6+7+4+3+0+6+7+2+5+6=49

Long(1)=9; long(2)=13, long(3)=9, long(4)=12, long(5)=9; long(6)=13

评分标准：总分 10 分，思路正确得 5 分，计算正确得 5 分，其它情况酌情扣分。

- 2、在 AVL 树中插入一个新结点后，怎么判断需要调整 AVL 树恢复平衡？怎样判断这种调整可以结束，不再需要进一步的调整？（5 分）

答：从新结点开始向上追溯，看新结点的双亲结点的平衡因子，如果变为 0，不需继续向上调整，如果变为 1，则需要继续向上追溯，如果变为 2，则需要做调整，因为调整后局部高度恢复到原来的值，所以不需要继续调整。

评分标准：叙述正确得 5 分，其它情况酌情扣分。

- 3、在  $n$  个不同元素的顺序表中找出最大和最小值需要至少进行多少次比较？算法的思路是怎样的？（6 分）

答： $3/2 * n - 2$

把表分为两部分，分别找出各部分的最大最小值，递归进行。

也可每次向前循环两个元素，对这两个元素比较一次，得到最大最小值，然后与前面得到的元素比较最大值和最小值。

评分标准：叙述正确得 4 分，计算正确得 2 分，其它情况酌情扣分。

- 4、对数据元素 1、13、29、18、75、60、43、54、90、46、21、34，采用散列函数  $\text{hash}(\text{key}) = \text{key} \% 13$  和线性探测再散列方法构造散列表 HT[13]。（1）请计算出各元素散列地址。（2）求搜索成功和不成功的平均搜索长度。（6 分）

答：各元素的散列地址为：1（1）、13（0）、29（3）、18（5）、75（10）、60（8）、43（4）、54（2）、90（12）、46（7）、21（9）、34（11）

搜索成功的平均搜索长度为： $(1 * 10 + 2 * 1 + 4 * 1) / 12 = 4/3$

搜索不成功的平均搜索长度为： $(0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12) / 13 = 6$

评分标准：散列地址计算正确、两种搜索长度计算正确，各得 2 分，其它情况酌情扣分。

- 5、若存于顺序存储结构中的  $n$  个元素的序列近似有序，即去掉其中少数  $k$  个元素的序列是有序序列，试对这种数据文件讨论用直接插入排序法、起泡排序法和直接选择排序法的渐进时间复杂度。（8 分）

答：对于基本有序的  $n$  个元素，用直接插入排序法其渐进时间复杂度为  $O(k*n)$ ，其中  $k$  为排列无序的元素个数， $n$  为排列有序的元素个数。显然，用直接插入法时，对有序元素的插入通常只需要比较 1 次即可确定其位置，而对无序元素的插入至多需要比较  $n$  次；也即，该方法的渐进时间复杂度为  $O(k*n)$ 。在用冒泡排序法、直接选择排序法对基本有序的  $n$  个元素排序时，其渐进时间复杂度都为  $O(n^2)$ 。

评分标准：总分 8 分，各种简单内排序方法分析比较正确或者部分正确为 1-8 分，否则为 0 分。

#### 四、算法题（35%）

（除写出相应算法之外，还需要说明算法和数据结构的设计思路，算法用 c 或者 C++ 语法描述）

- 1、设  $T$  是一棵满二叉树，写一个递归算法，把  $T$  的前序遍历序列转换成后序遍历序列。（10 分）

答：由于是满二叉树，结点总数为  $2n+1$ ，先序的第一个结点为根结点，第 2 到  $n+1$  为左子树的结点，第  $n+2$  到  $2n+1$  为右子树的结点。假设已经有了一个先序数组  $pre[a1,...,b1]$ ，求后序数组  $post[a2,...,b2]$  的算法如下：

```
convert(int[] pre, int[] post, int preStart, int postStart, int k) {
    post[postStart+k] = pre[preStart];
    if(k==0) return;
    convert(pre, post, preStart+1, postStart+k/2+1, k/2);
    convert(pre, post, preStart+k/2+1, postStart, k/2);
    return;
}
```

**评分标准：**总共 10 分。

思路 3 分——思路清晰正确为 3 分，方法不限于参考答案这一种，思路不完全正确最多为 1 分，没写为 0 分；

程序 7 分——程序结构基本正确为 3 分，部分正确为 5 分，全部正确为 7 分。

**注：**由于算法题可能会存在多种求解方法，因此评分过程可依据实际情况进行动态调整。

- 2、设  $A$ 、 $B$  是长度为  $n$  的两个非降序数组。如果将这  $2n$  个数全体排序，处于位置  $n$  的数称为中位数。设计一个最坏情况下  $O(\log n)$  的算法，求这个中位数。并且证明你的算法的时间复杂性。（15 分）

答：可以递归地完成。首先对于两个长度为  $k$  的有序表，可以比较它们各自的中位数  $A[k/2]$  和  $B[k/2]$ ，比较结果分为 3 种情况：

- (1) 如果  $A[k/2] > B[k/2]$ , 则  $A[k/2]..A[k]$  都大于  $B[0]..B[k/2-1]$ , 再加上  $B[k/2]..B[k]$  本来就都大于  $B[0]..B[k/2-1]$ , 多于  $k$  个数比  $B[0]..B[k/2-1]$  大, 所以, 第  $k$  大的数不可能在  $B[0]..B[k/2-1]$  中, 因此可以将  $B[0]..B[k/2-1]$  这部分丢掉; 类似地, 第  $k$  大的数也不可能在  $A[k/2+1]..A[k]$  中, 所以也可以将这部分丢掉, 因此得到了两个新的序列:  $A[0]..A[k/2]$  和  $B[k/2]..B[k]$ , 然后继续递归。
  - (2) 如果  $A[k/2] < B[k/2]$ , 这种情况和第 (1) 种情况类似, 只不过丢掉的是  $A[0]..A[k/2-1]$  和  $B[k/2+1]..B[k]$ , 留下的是  $A[k/2]..A[k]$  和  $B[0]..B[k/2]$ 。
  - (3) 如果  $A[k/2] == B[k/2]$ , 因为  $A[k/2]$  大于  $A[0]..A[k/2-1]$  和  $B[0]..B[k/2-1]$ , 并且  $A[k/2] == B[k/2]$ , 所以第  $k$  大的数就是  $A[k/2]$ 。
- 因为每次比较都能减少一半的元素, 所以最坏情况下递归深度为  $\log n$ , 所以算法的时间复杂度为  $\log n$ 。

算法如下, 假设  $A$  和  $B$  为整型数组:

```
int findMid(int[] a, int[] b, int startA, int startB, int k) {
    if(a[startA+k/2] == b[startB+k/2]) return a[startA+k/2];
    if(a[startA+k/2] > b[startB+k/2]) findMid(a,b,startA+k/2,startB,k/2);
    else (a[startA+k/2] < b[startB+k/2]) findMid(a,b,startA,startB+k/2,k/2);
}
```

**评分标准:** 总共 15 分。

思路 5 分——思路清晰正确为 5 分, 方法不限于参考答案这一种, 思路不完全正确最多为 2 分, 没写为 0 分;

程序 10 分——程序结构基本正确为 5 分, 部分正确为 7 分, 全部正确为 10 分。

**注:** 由于算法题可能会存在多种求解方法, 因此评分过程可依据实际情况进行动态调整。

- 3、设计一个算法, 判断一个有向无环图中是否存在这样的顶点, 该顶点到其它任意顶点都有一条有向路。(10 分)

答: 如果存在这样的顶点, 由于是有向无环图, 则该顶点到其它顶点的路径形成一棵树, 树上的结点数量等于图上的结点数量。因此依次从每个结点开始, 用深度优先或者广度优先, 如果遍历到的结点数量等于图上的结点数量, 则存在, 否则不存在。

如果用广度优先, 图用邻接表表示, 其数据结构描述如下:

```
struct node {
    int adjvex;
    node *next;
}
```

```
struct g {
    vertextype data;
    node *firstarc;
}typedef ADJLIST;
```

```
int rbfs(ADJLIST[] g, int v, int n) {
```

```

int i, count, yes;
yes = 0;
count = 1;
Queue Q;
for(i=0; i<n; i++) visited[i] = 0;
ENQUEUE(v, Q);
visited[v] = 1;

while(!EMPTY(Q)&&!yes) {
    w=Q.front;
    p=g[w].firstarc;
    while((p!=NULL) &&!yes) {
        w=p->adjvex;
        if(visited[w] = 0) {
            visited[w] = 1;
            count++;
            ENQUEUE(w,Q);
        }
        else p=p->next;
    }
}
if(count == n) yes=1;
return yes;
}

Bool  mainLoop(ADJLIST[] g, int n) {
    for(int i=0;i<n;i++)  if( rbfs(g, i, n) == 0) return true;
    return false;
}

```

**评分标准：** 总共 10 分。

思路 2 分——思路清晰正确为 2 分，方法不限于参考答案这一种，思路不完全正确最多为 1 分，没写为 0 分；

关键数据结构 2 分——如邻接表中顶点结点与边结点；

程序 6 分——程序结构基本正确为 2 分，部分正确为 4 分，全部正确为 6 分。

**注：** 由于算法题可能会存在多种求解方法，因此评分过程可依据实际情况进行动态调整。

# 复旦大学计算机科学技术学院

## 《数据结构》期末考试试卷（参考答案与评分标准）

### A 卷 共 8 页

课程代码: COMP130004.01-03 考试形式: ☐开卷 ☒闭卷 2012 年 1 月  
(本试卷答卷时间为 120 分钟, 答案必须写在试卷上, 做在草稿纸上无效)

专业\_\_\_\_\_学号\_\_\_\_\_姓名\_\_\_\_\_成绩\_\_\_\_\_

| 题号 | 一 | 二 | 三 | 四 | 总分 |
|----|---|---|---|---|----|
| 得分 |   |   |   |   |    |

#### 一、填空题 (25%,1~8 题每题 2 分)

- 1、 设 W 为一个二维数组, 其每个数据元素占用 4 个字节, 行下标 i 从 0 到 7 , 列下标 j 从 0 到 3 , 则二维数组 W 的数据元素共占用\_\_\_\_\_个字节。若按行顺序存放二维数组 W, 其起始地址为 100(10 进制), 则二维数组元素 W[6, 3]的起始地址为\_\_\_\_\_ (10 进制表示)。

答: 128, 208。

- 2、 后缀算式  $9\ 2\ 3\ +\ -\ 10\ 2\ /\ -$  的值为\_\_\_\_\_。中缀算式  $(3+4X) - 2Y/3$  对应的后缀算式为\_\_\_\_\_。

答: -1,  $3\ 4\ X\ * +\ 2\ Y\ * 3\ /\ -$ 。

- 3、 由权值分别为 11, 8, 6, 2, 5 的叶子结点生成一棵哈夫曼树, 它的带权路径长度为\_\_\_\_\_。

答: 71

- 4、 在有序表 (12, 24, 36, 48, 60, 72, 84) 中二分查找关键字 72 时所需进行的关键字比较次数为\_\_\_\_\_。

答: 2

- 5、 在含 n 个顶点和 e 条边的无向图 (无自环、重边) 的邻接矩阵中, 零元素的个数为\_\_\_\_\_。

答:  $n^2-2e$

- 6、 已知一棵完全二叉树中共有 768 结点, 则该树中共有\_\_\_\_\_个叶子结点。

答: 384

- 7、 有向图的边集为{(a, c), (a, e), (e, b), (e, d), (b, d), (d, c), (c, f)}, 该图的一个拓扑排序为:\_\_\_\_\_

答: a e b d c f

- 8、 当输入序列局部有序或元素个数较小时, 在快速排序、选择排序、插入排序、归并排序、堆

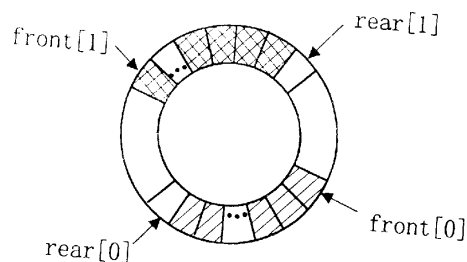
排序中，最佳的排序方法是\_\_\_\_\_。

答：插入排序

9、假设两个队列共享一个循环向量空间（参见右图），

其类型 Queue2 定义如下：

```
typedef struct{
    DataType data[MaxSize];
    int front[2],rear[2];
}Queue2;
```



对于  $i=0$  或  $1$ ,  $front[i]$ 和  $rear[i]$ 分别为第  $i$  个队列的头指针和尾指针。请对以下算法填空，实现第  $i$  个队列的入队操作。

```
int EnQueue (Queue2 *Q, int i, DataType x)
{//若第 i 个队列不满，则元素 x 入队列，并返回 1；否则返回 0
    if( $i < 0 || i > 1$ )return 0;
    if( $Q \rightarrow rear[i] == Q \rightarrow front[_____]$ ) return 0;
     $Q \rightarrow data[_____] = x$ ;
     $Q \rightarrow rear[i] = [_____]$ ;
    return 1;
}
```

答： $(i+1)\%2$  (或  $1-i$ );  $Q \rightarrow rear[i]$ ;  $(Q \rightarrow rear[i]+1)\%Maxsize$ ; (每空 1 分)

10、以下是一个判断二叉树是否平衡的程序，对于平衡的二叉树， $depth$  将会返回其高度（对于非平衡二叉树不要求返回）。请在空白处填上代码完成程序。（6 分）

```
bool isBalanced( BiTreeNode * pRoot, int& depth)
{
    if( $pRoot == NULL$ ) {_____; return _____;}
    int leftDepth, rightDepth;
    if(_____)
    {
        int dif = leftDepth - rightDepth;
        if(_____)
        {
             $depth = (1+leftDepth) > (1+ rightDepth) ? 1+leftDepth : 1+ rightDepth$ ;
            return _____;
        }
    }
    return false;
}
```

}

答: depth = 0 (1分) true(1分)

isBalanced(pRoot->left, leftDepth) && isBalanced(pRoot->right, rightDepth) (2分)

dif >= -1 && dif <= 1 (1分)

true (1分)

## 二、选择题 (10%)

1、对于双向循环链表,每个结点有两个指针域 next 和 prior,分别指向前驱和后继。在 p 指针所指向的结点之后插入 s 指针所指结点的操作应为 ( )

A. p->next = s; s->prior = p; p->next->prior = s; s->next = p->next;

B. p->next = s; p->next->prior = s; s->prior = p; s->next = p->next;

C. s->prior = p; s->next = p->next; p->next = s; p->next->prior = s;

D. s->prior = p; s->next = p->next; p->next->prior = s; p->next = s;

答: D。

2、一个栈的输入序列为 1 2 3,则下列序列中不可能是栈的输出序列的是( )

A. 2 3 1

B. 3 2 1

C. 3 1 2

D. 1 2 3

答: C。

3、采用开放定址法处理散列表的冲突时,其平均查找长度 ( )。

A. 低于链接法处理冲突

B. 高于链接法处理冲突

C. 与链接法处理冲突相同

D. 高于二分查找

答: B。

4、假设一个有 n 个顶点和 e 条弧的有向图用邻接表表示,则删除与某个顶点  $v_i$  相关的所有弧的时间复杂度是( )

A.  $O(n)$

B.  $O(e)$

C.  $O(n+e)$

D.  $O(n*e)$

答: C。

5、设有 6 个结点的无向图,该图至少应有( )条边才能确保是一个连通图。

A.5

B.6

C.7

D.8

答: A

## 三、问答题 (40%)

1、设一棵 m 叉树中有  $N_1$  个度数为 1 的结点,  $N_2$  个度数为 2 的结点, ...,  $N_m$  个度数为 m 的结点,则该树中共有多少个叶子结点? (请写出求解过程) (5 分)

**答：**度数为  $i$  的结点的孩子个数为  $i \times N_i$ 。除了根结点，其他结点都是某结点的孩子，且父亲结

点唯一。因此该树共有  $\text{total} = 1 + \sum_{i=1}^m i \times N_i$  个结点。

度数大于 0 的结点都是非叶子结点，因此内部结点个数  $\text{inner} = \sum_{i=1}^m N_i$ 。

叶子结点个数  $\text{leaf} = \text{total} - \text{inner} = 1 + \sum_{i=1}^m (i-1) \times N_i = 1 + \sum_{i=2}^m (i-1) \times N_i$

**评分标准：**总分 5 分，分析过程正确或者部分正确为 1-5 分，否则为 0 分。

2、一个线性表为  $B = (12, 23, 45, 57, 20, 03, 78, 31, 15, 36)$ ，设散列表为  $\text{HT}[0..12]$ ，散列函数为  $H(\text{key}) = \text{key} \% 13$  并用线性探查法解决冲突，请画出散列表，并计算等概率情况下查找成功的平均查找长度。（5 分）

答：

| 0  | 1 | 2  | 3  | 4 | 5  | 6  | 7  | 8  | 9 | 10 | 11 | 12 |
|----|---|----|----|---|----|----|----|----|---|----|----|----|
| 78 |   | 15 | 03 |   | 57 | 45 | 20 | 31 |   | 23 | 36 | 12 |

查找成功的平均查找长度： $\text{ASL}_{\text{succ}} = 14/10 = 1.4$

（散列表 4 分，查找成功的平均查找长度 1 分）

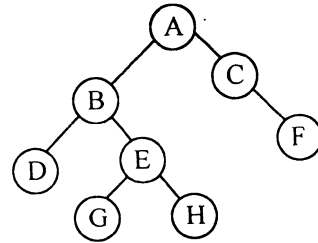
3、已知二叉树的存储结构为二叉链表，阅读下面算法。

```
typedef struct node {
    DataType data;
    Struct node * next;
}ListNode;
typedef ListNode * LinkList;
LinkList Leafhead = NULL;
Void Inorder (BinTree T){
    LinkList s;
    If(T){
        Inorder(T->lchild);
        If ((!T->lchild)&&(!T->rchild)){
```

```

s=(ListNode*)malloc(sizeof(ListNode));
s->data=T->data;
s->next=Leafhead;
Leafhead=s;
}
Inorder(T->rchild);
}

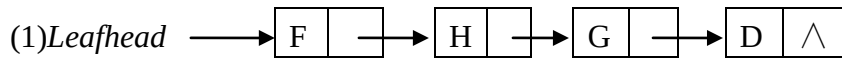
```



对于如下所示的二叉树

- (1) 画出执行上述算法后所建立的结构；(4 分)
- (2) 说明该算法的功能。(2 分)

答:

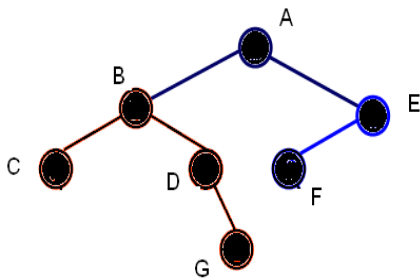


(2) 中序遍历二叉树，按遍历序列中叶子结点数据域的值构建一个以 Leafhead 为头指针的逆序单链表（或按二叉树中叶子结点数据自右至左链接成一个链表）。

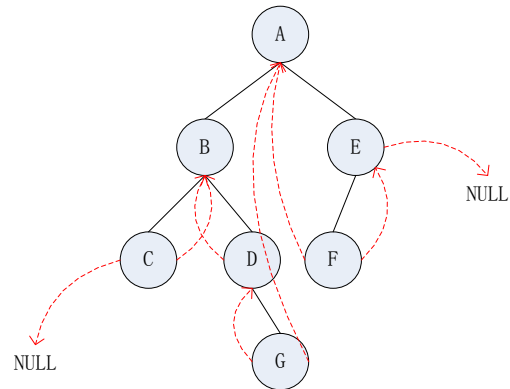
- 4、一棵二叉树的先序序列为 ABCDGEF，中序序列为 CBDGAFE。请画出该二叉树并将其中序线索化，后将二叉树转换为相应的森林。(5 分)

答:

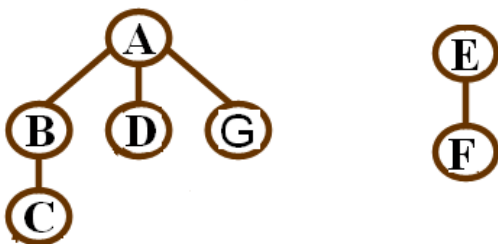
(二叉树: 1 分):



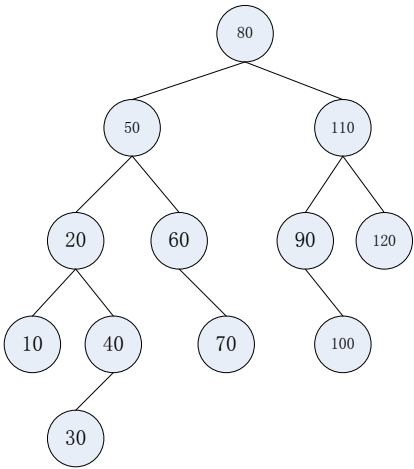
(中序线索: 2 分):



(森林: 2 分):

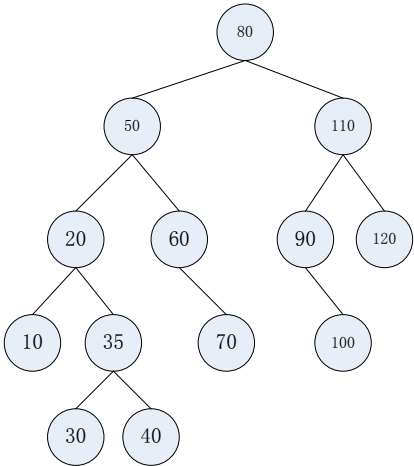


5、对于以下 AVL 树，依次插入关键码 35、28、5。请画出每插入一个关键码之后 AVL 树的形状。  
(6 分)

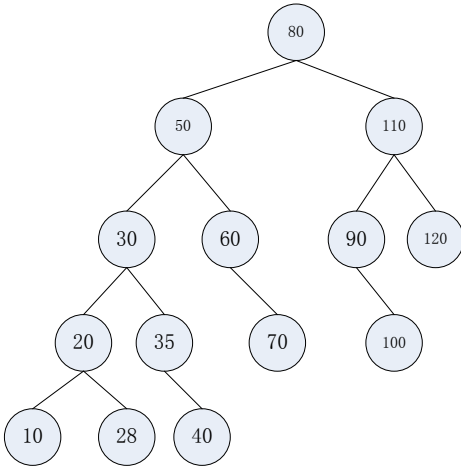


答:

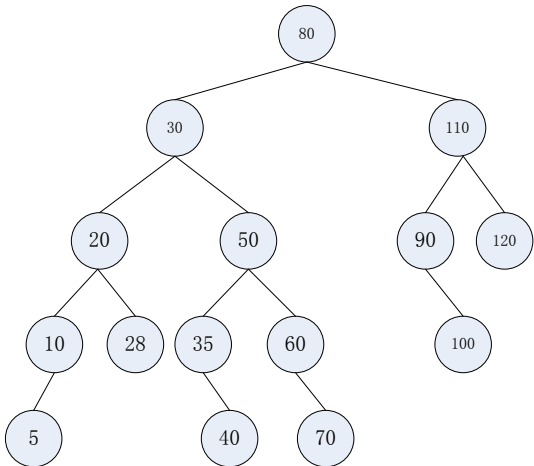
插入 35 (2 分):



插入 28 (2 分):



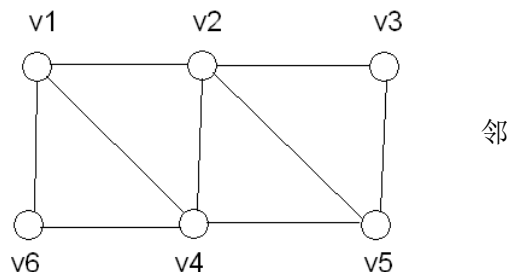
插入 5 (2 分):



**评分标准：**总分 6 分，每次插入后的形状 2 分。

6、已知一个连通图如下图所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点 v1 出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。(8 分)

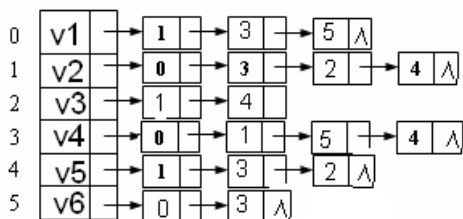
- (1) 图的邻接矩阵 (1 分)
- (2) 邻接表存储示意图 (1 分)
- (3) 从 v1 开始的深度优先遍历的顶点序列 (2 分)
- (4) 分析在深度遍历过程中，分别使用邻接矩阵和接表存储的算法复杂度 (2 分)
- (6) 讨论在图遍历问题中，这两种存储方式的优劣。(2 分)



**答：**(1)图的邻接矩阵 (1 分)

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

(2) 邻接表存储示意图 (1 分)



(3) 深度优先遍历的顶点序列: v1,v2,v3, v5,v4, v6 (2 分) (有多个答案)

(4) 深度优先邻接矩阵复杂度:  $O(n^2)$ , n 为顶点数; 邻接表复杂度:  $O(n+e)$ , n 为顶点数, e 为边的条数。(2 分)

(5)在图的遍历算法中，遍历一个顶点的邻居，对于邻接矩阵，时间复杂度为  $O(n)$ ，而邻接表只需要该顶点的邻居数量复杂度；因此对于整个图的遍历，如深度优先、广度优先等，邻接矩阵需要  $O(n^2)$ ，而邻接表时间复杂度为  $O(n+e)$ ，当图为稀疏图时，邻接表的时间复杂度优于邻接矩阵，而且存储开销也优于邻接矩阵；当图为密集图时，两者性能差不多；总的来说，邻接表的时间复杂度和空间存储开销都优于邻接矩阵。(2 分)

7、分析对比 AVL 树和 Hash 的时空特性，并讨论它们的适合场合。(6 分)

**答：**

时空特性:

AVL 树是高度平衡的二叉查找树, 查找时间复杂度为  $O(\log n)$ , Hash 的查找时间复杂度为  $O(1)$ 。  
存储开销 Hash 往往比 AVL 树小。

适合场合:

Hash 必须处理冲突, 而 AVL 树不存在这种问题。对于删除操作, AVL 树的时间开销很稳定, 为  $O(\log n)$ , 而 Hash, 如果采用拉链法处理冲突, 则删除操作易于实现, 如果采用开放定址法, 则删除操作很难实现, 因此开放定址法的 Hash 不适合更新频繁的数据存储。另外, AVL 树对数据的存储是有序的, 而 Hash 对数据的存储并不能反映数据之间的大小关系。因此, AVL 树适用于有序的数据存储, 而 Hash 适用于数据量比较大且分布比较均匀, 对数据排序无要求的数据存储。

#### 四、算法题 (25%, 第一题要求写代码; 后两题要求写伪代码或步骤清晰的解题思路)

1、堆是一种很常用的数据结构, 其中的一个重要用途是优先队列。以整数最小堆为例, 写出优先队列 (最小优先) 的两个基本操作的代码: (10 分)

(1) 入队, 即插入一个元素到存放堆的数组的末尾, 然后调整堆。

(2) 出队, 即删除堆顶节点, 然后调整堆。

void insert( int k, int a[], int n ) { // 假设新的元素 k 先被放在堆末尾位置 a[n] 单元中

```
    答: n++;
        int j = n-1;
        int i = (j-1)/2;
        while(j > 0)
        {
            if(a[i] <= k)
                break;
            a[j] = a[i];
            j = i;
            i = (i-1)/2;
        }
        a[j] = k;
```

}

int deleteMin( int a[], int n ) { // 假设在堆顶元素被删除之前, 堆中共有 n 个元素

```
    答: a[0] = a[n-1];
        n--;
        int i = 0;
        int temp = a[i];
        while((j = 2*i + 1) < n)
        {
            if( j < n-1 && a[j+1] < a[j])
                j++;
            if(a[j] >= temp)
                break;
            a[i] = a[j];
            i = j;
```

```

    }
    a[i] = temp;
}

```

- 2、堆的另外一个常用场合是求 top k 问题。假设我们已经有一个 n 个元素的最小堆，如果用最直观的方法求最小的 k 个元素，其计算复杂度为  $O(k\log n)$ 。试问：如何在更小的复杂度内求 top k 问题？用伪代码描述你的方法，并给出复杂度证明。(7 分)

**答：**

假设原最小堆为 H，新建一个最小堆 H'，H'初始化为空。

将最小堆 H 的堆顶元素 a 插入 H'；然后以下步骤执行 k 次：

将 H'的堆顶元素 a'删除并输出，然后将 a'在 H 中的两个孩子插入 H'中。

为了快速找到 a'在 H 中的两个孩子，H'存储的实际上是元素在 H 中的下标，而大小比较通过下标读取 H 中对应的元素。

由于 k 次对 H'的删除操作，最多将 2 个新元素插入 H'中，因此 H'的大小不超过 2k。每次插入删除操作复杂度都为  $O(\log 2k)$ ，因此算法复杂度为  $O(k\log k)$ 。

**评分标准：满分：** $O(k\log k)$ 或者更好。若提出了快速排序的 partition 函数 ( $O(n)$  复杂度)，给 4 分。

- 3、给定一个无向图  $G(V, E)$ ，V 为顶点集合，E 为边集合。对于 V 中的两个顶点 u 和 v，如果图中存在一条 u 到 v 的路径，则称 u 和 v 彼此可达。设计一种数据结构，将图 G 进行预处理得到该数据结构，使得对于任意 u 和 v，通过该数据结构可以在常数时间得到 u 和 v 是否彼此可达。要求该数据结构存储开销为  $O(|V|)$ ，预处理复杂度为  $O(|V| + |E|)$ 。(8 分)

注意——图中的结点从 0 开始用整数连续编号。图 Graph 上的操作有：取得结点总数 `int GetVertexNum()`；取得 u 之后的 v 的下一个邻接顶点 `int GetNextNeighbor(int v,int u)`，u 为-1 则取得 v 的第一个邻接顶点，返回-1 表示没有下一个邻接顶点了；可以直接使用数组、链表、栈和队列这些基本数据结构及其上的基本操作，但要注释每个所使用操作的含义。

- (1) 描述所设计的数据结构，描述如何得到该数据结构，并写出预处理的代码。
- (2) 描述如何通过该数据结构在常数时间内得到 u 和 v 是否可达，并写出相应代码。

**答：** 本题考察学生对连通分量的掌握和运用。使用 BFS 算法得到图的 N 个连通分量，为每个连通分量分配一个 id。用一个包含 |V| 个元素的数组 `connectedCom[|V|]`，记录每个顶点属于的连通分量 id，即 `connectedCom[u]` 表示 u 属于的联通分量。对于 u 和 v，若 `connectedCom[u] = connectedCom[v]`，则 u 和 v 彼此可达。

**评分标准：** 本题答案不唯一，可以是 BFS、DFS，也可以是并查集。

总共 8 分。

思路 2 分——思路清晰正确为 2 分，方法不限于参考答案这一种，思路不完全正

确最多为 1 分，没写为 0 分；算法 1-6 分。